

Ministère de l'Enseignement Supérieur et de la
Recherche Scientifique
Direction Générale des Études Technologiques
Institut Supérieur des Études Technologiques
de Bizerte
Département des Technologies de l'Information



Référence	
Dép.	TI
Année	2026
N°	SEM2615

Rapport de Projet de Fin d'Études

Présenté en vue de l'obtention du :

Licence Appliquée en Systèmes Embarqués et Mobiles

**Marché Artisanal Tunisien : Valorisation du
Patrimoine
(Marchi Marketplace)**

Encadré par :

M. Hatem HAMDY (ISET Bizerte)

M. Mohamed RABOUDI (Alder Green Solutions)

Préparé par :

Houssem JEMAI

Réalisé au sein de :

Entreprise : Alder Green Solutions

Adresse : Tunis, Tunisie

Email : contact@alder.earth

Année Universitaire : 2025 / 2026

Dédicace

*À mon père,
Fawzi Jemai*

*Tu n'es plus là pour voir l'aboutissement de ce
travail,
mais ta présence reste gravée dans chaque effort,
chaque nuit de veille et chaque ligne de ce
rapport.
Tu m'as appris la rigueur, la dignité et le
courage.*

*Que Dieu t'accorde Sa miséricorde et t'accueille
en Son paradis.*

*À ma mère, **Rebeh Maalaoui**,
source de force, de patience et d'amour
inconditionnel,
dont les sacrifices silencieux ont rendu ce travail
possible.*

*À ma tante, **Amina Jemai**,
pour son soutien constant et sa présence
réconfortante
à chaque étape de ce parcours.*

*À mon frère, **Dhafer Jemai**,
pour son professionnalisme, ses conseils éclairés
et la confiance qu'il m'a toujours accordée.*

Remerciements

Je remercie vivement **M. Hatem Hamdi**, mon encadrant pédagogique, pour ses conseils stratégiques, sa disponibilité et sa relecture technique tout au long de ce projet. Mes remerciements vont également à **M. Mohamed Raboudi** (Alder Green Solutions), encadrant professionnel, pour le contexte métier, la vision de valorisation de l'économie artisanale tunisienne, et la confiance accordée dans la conduite autonome du développement.

Je remercie enfin mes collègues pour les discussions constructives, et l'ensemble du corps enseignant pour la qualité de la formation reçue.

Ce projet est une démonstration concrète que la transformation numérique locale est possible, même avec des ressources limitées, lorsqu'on applique rigoureusement les bonnes pratiques d'ingénierie logicielle : Scrum, TDD, SOLID, sécurité par conception.

Table des matières

Dédicace	2
Remerciements	3
Liste des figures	7
Liste des tableaux	8
Liste des Abréviations	9
Introduction Générale	10
Chapitre I : Cadre Général du Projet	11
Introduction	11
1 Présentation de l'organisme d'accueil	11
2 Présentation du projet Marchi	11
3 Étude de l'existant	12
4 Critique de l'existant	12
5 Solution proposée	13
6 Méthodologie adoptée	13
7 Outils de gestion	14
Conclusion	14
Chapitre II : Étude Comparative et Choix Techniques	15
Introduction	15
1 Analyse des compétiteurs majeurs	15
2 Justification des choix techniques	16
a) Backend : Spring Boot 3.2 vs Node.js	16
b) Frontend Mobile : Flutter 3.3+ vs React Native	17
c) Base de données : MySQL 8.x vs MongoDB	17
d) Justification des services tiers	18
3 Environnement de développement	18
Conclusion	19
Chapitre III : Analyse des Besoins et Conception	20
Introduction	20
1 Spécification des besoins – Épics	20
2 Diagramme de cas d'utilisation	21
3 Backlog du produit	22
4 Planification des sprints	23

Conclusion	24
Chapitre IV : Architecture Générale	25
Introduction	25
1 Architecture globale du système	25
2 Architecture Backend Spring Boot	26
3 Architecture Firebase Cloud Messaging	27
4 Workflow des notifications push	27
5 Diagramme de déploiement UML	28
6 Schéma Entité-Relation (ERD)	29
Conclusion	30
Chapitre V : Sprint 1 – Authentification et Sécurité	31
Introduction	31
1 Sprint Backlog	31
2 Flux d'inscription et de vérification	31
3 Diagramme de séquence – Connexion JWT	31
4 Réalisations et captures d'écran	32
5 Tests Sprint 1	33
Conclusion	33
Chapitre VI : Sprint 2 – Gestion des Utilisateurs et Adresses	34
Introduction	34
1 Sprint Backlog	34
Conclusion	35
Chapitre VII : Sprint 3 – Catalogue et Gestion des Produits	36
Introduction	36
1 Sprint Backlog	36
Conclusion	38
Chapitre VIII : Sprint 4 – Expérience Marketplace et Panier	39
Introduction	39
1 Sprint Backlog	39
Conclusion	40
Chapitre IX : Sprint 5 – Commandes et Paiement Cash on Delivery	41
Introduction	41
1 Sprint Backlog	41
Conclusion	42

Chapitre X : Sprint 6 – Système de Livraison CP1	43
Introduction	43
1 Sprint Backlog	43
Conclusion	44
Chapitre XI : Sprint 7 – Administration, Notifications et Finitions	45
Introduction	45
1 Sprint Backlog	45
2 Conception du système de notifications	46
Conclusion	48
Chapitre XII : Sécurité, Tests et Déploiement	50
Introduction	50
1 Sécurité JWT et RBAC	50
2 Vérification HMAC-SHA256 des bons CP1	51
3 Stratégie de test	53
a) Exemples de tests unitaires	53
4 Déploiement sur Railway	54
5 Analyse critique et perspectives	56
a) Limites techniques identifiées	56
b) Perspectives d'amélioration	56
Conclusion	57
Conclusion Générale	58
Bibliographie	60
Annexe A : Endpoints API	61

Liste des figures

III.1 Diagramme de cas d'utilisation – Système Marchi Marketplace	22
IV.1 Architecture globale en couches de l'application Marchi	25
IV.2 Architecture Backend Spring Boot – couches et services tiers	26
IV.3 Architecture du système de notifications Firebase Cloud Messaging	27
IV.4 Diagramme de déploiement UML – Infrastructure Marchi	29
IV.5 Schéma Entité-Relation (ERD) – Base de données Marchi	30
V.1 Diagramme de séquence – Flux de connexion JWT avec gestion des erreurs	32
V.2 Écran de connexion Flutter	33
V.3 Écran d'inscription Flutter	33
VI.1 Édition profil artisan	35
VI.2 Saisie d'adresse avec détection automatique du code postal	35
VII.1 Diagramme de classes – Module Produit	37
VII.2 Formulaire création produit	38
VII.3 Upload d'image produit via Cloudinary	38
VIII. Page d'accueil marketplace	40
VIII. Écran panier	40
IX.1 Diagramme d'états d'une commande Marchi	41
IX.2 Écran confirmation commande COD	42
IX.3 Historique commandes client	42
X.1 Génération bon CP1 dans l'application	44
X.2 Aperçu PDF bon de livraison CP1	44
XI.1 Tableau de bord ARTISAN	46
XI.2 Centre de notifications in-app (lu / non lu)	46
XI.3 Diagramme de classes – Module <code>NotificationAuditService</code> et entité <code>NotificationHistory</code>	47
XI.4 Diagramme de séquence – <code>NotificationAuditService.save()</code> vers MySQL	48
XI.5 Diagramme d'états – Cycle de vie d'une <code>NotificationHistory</code>	48
XII.1 Pyramide des tests – Stratégie de test Marchi	53
XII.2 Processus de déploiement Railway – Vue synthétique	55

Liste des tableaux

.1	Abréviations et acronymes utilisés dans le rapport	9
I.1	Environnement technique du projet Marchi	12
I.2	Axes techniques de la solution Marchi	13
II.1	Analyse comparative des compétiteurs (Jumia, Etsy, Facebook Marketplace, Marchi)	16
II.2	Comparaison backend : Spring Boot 3.2 vs Node.js / Express	17
II.3	Comparaison frontend mobile : Flutter 3.3+ vs React Native	17
II.4	Outils de développement du projet Marchi	19
III.1	Spécification des épics fonctionnels (EP-01 à EP-08)	20
III.2	Backlog du produit – extrait des user stories par épique	23
III.3	Planification des 7 sprints sur 13 semaines	24
IV.1	Responsabilités des couches de l’architecture Spring Boot	27
IV.2	Types de notifications push implémentées dans Marchi	28
V.1	Résultats des tests – Sprint 1 : Authentification et sécurité	33
X.1	Résultats des tests – Sprint 6 : Livraison CP1 et génération PDF	44
XII.1	RBAC – Rôles, droits et exemples d’endpoints	51
XII.2	Résultats des tests – Endpoint scan QR (HMAC-SHA256)	52
XII.3	Métriques de couverture des tests unitaires par sprint	54
XII.4	Tests de validation de l’APK Android avant distribution	56
XII.5	Perspectives d’amélioration envisagées à 3–6 mois	57
XII.6	Endpoints API – Marchi Marketplace	61

Liste des Abréviations

Le tableau suivant recense l'ensemble des sigles et acronymes utilisés dans ce rapport, accompagnés de leur signification complète.

Tableau .1 : Abréviations et acronymes utilisés dans le rapport

Abréviation	Signification
API	Application Programming Interface
APK	Android Package Kit
ACID	Atomicity, Consistency, Isolation, Durability
CDN	Content Delivery Network
COD	Cash on Delivery (Païement à la livraison)
CP1	Format de bon de livraison postal tunisien
CRUD	Create, Read, Update, Delete
FCM	Firebase Cloud Messaging [3]
HMAC	Hash-based Message Authentication Code
JPA	Java Persistence API
JWT	JSON Web Token [4]
MENA	Middle East and North Africa
ORM	Object-Relational Mapping
OTP	One-Time Password
PaaS	Platform as a Service
PFE	Projet de Fin d'Études
RBAC	Role-Based Access Control
REST	Representational State Transfer
SDK	Software Development Kit
TDD	Test-Driven Development
TTL	Time To Live
UML	Unified Modeling Language

Introduction Générale

L'artisanat tunisien constitue un pilier fondamental de l'identité culturelle et économique du pays. Des poteries de Nabeul aux tapis berbères de Gafsa, en passant par les bijoux en argent de Djerba et les tissus de soie de Tunis, ce savoir-faire millénaire représente une richesse patrimoniale considérable. Pourtant, les artisans qui perpétuent ces traditions souffrent d'un accès limité aux marchés modernes et peinent à s'appuyer sur des outils numériques adaptés à leurs besoins spécifiques.

Le commerce électronique connaît une croissance rapide à l'échelle mondiale, et la Tunisie ne fait pas exception à cette tendance. Cependant, les plateformes disponibles sur le marché local restent inadaptées au contexte des artisans : elles n'intègrent pas le système logistique postal tunisien (format CP1), ne proposent pas de profils vendeur dédiés, et s'appuient sur des modes de paiement insuffisamment adaptés à une population encore peu bancarisée pour le commerce en ligne.

C'est dans ce contexte que s'inscrit le projet **Marchi**, développé dans le cadre du stage de fin d'études au sein d'Alder Green Solutions, entreprise spécialisée dans la transition écologique et la numérisation de l'économie locale tunisienne. Marchi est une application marketplace mobile-first, conçue spécifiquement pour connecter artisans et acheteurs, dans le respect des contraintes techniques et culturelles du marché tunisien.

Ce rapport est structuré en **onze chapitres** suivant la méthodologie Scrum adoptée :

- Le **Chapitre I** présente le cadre général du projet, l'organisme d'accueil, l'étude de l'existant et la solution proposée.
- Le **Chapitre II** expose l'étude comparative des solutions existantes et justifie les choix techniques retenus.
- Le **Chapitre III** détaille l'analyse des besoins, le backlog produit et la planification des sprints.
- Le **Chapitre IV** décrit l'architecture globale du système.
- Les **Chapitres V à XI** documentent chacun un sprint Scrum : authentification, gestion des utilisateurs, catalogue produits, expérience marketplace, commandes COD, livraison CP1, et administration/notifications.
- Le **Chapitre XII** traite des aspects transversaux : sécurité avancée, stratégie de test et déploiement en production.

Chapitre I :

Cadre Général du Projet

Introduction

Ce chapitre présente le cadre général dans lequel s'inscrit le projet Marchi. Nous exposons successivement l'organisme d'accueil, le contexte du projet, l'analyse de l'existant et ses limites, la solution proposée, la méthodologie adoptée et les outils de gestion retenus. L'objectif est de situer le projet dans son environnement professionnel et académique avant d'aborder les aspects techniques.

1. Présentation de l'organisme d'accueil

Alder Green Solutions est une entreprise innovante de la région MENA spécialisée dans la transition écologique du secteur agricole. Son activité principale couvre le développement de systèmes agricoles durables, les solutions de neutralité carbone (*Carbon Direct Removals*), ainsi que la numérisation de l'économie locale tunisienne. C'est dans cette dernière dimension que s'inscrit le projet Marchi : en valorisant le savoir-faire artisanal tunisien par le numérique, le projet contribue directement à la vision de création de revenus durables pour les producteurs et artisans locaux, sous l'encadrement de **M. Mohamed Raboudi**.

- Développement de systèmes agricoles durables et écologiques
- Solutions *Carbon Direct Removals* pour la neutralité carbone
- Numérisation et valorisation de l'économie locale tunisienne
- Création de revenus durables pour les producteurs et artisans locaux

2. Présentation du projet Marchi

Marchi est une application marketplace mobile-first dédiée à la commercialisation du savoir-faire artisanal tunisien. Elle vise à connecter directement les artisans aux acheteurs, en proposant une expérience adaptée aux contraintes du marché local : paiement à la livraison (COD), intégration native du format postal CP1, et sécurité robuste par JSON Web Token.

Le projet a été conduit en **7 sprints Scrum sur 13 semaines**, suivant une dé-

marche agile permettant des livraisons incrémentales et une adaptation continue aux retours d'utilisation. La table I.1 présente l'environnement technique retenu.

Le tableau I.1 présente l'environnement technique retenu pour le développement du projet Marchi, couvrant les deux couches principales (backend Spring Boot et frontend Flutter), la base de données, les services tiers et les outils de déploiement.

Tableau I.1 : Environnement technique du projet Marchi

Couche	Technologie	Version
Backend API	Spring Boot	3.2.x
Frontend mobile	Flutter	3.3+
Base de données	MySQL	8.x
ORM	Spring Data JPA	–
Gestion d'état	Riverpod	2.x
Génération PDF	iTextPDF	5.x
Sécurité	Spring Security + JWT	–
Notifications	Firebase FCM	–
Images CDN	Cloudinary [5]	SDK Java
Déploiement	Railway (PaaS)	–
CI/CD	GitHub Actions	–

3. Étude de l'existant

Le marché tunisien du commerce en ligne dispose de plusieurs plateformes génériques (Jumia, Facebook Marketplace, sites e-commerce locaux). Une analyse préliminaire de ces solutions révèle des caractéristiques communes qui les rendent inadaptées au profil spécifique des artisans tunisiens :

- Plateformes conçues pour un public généraliste, sans profil vendeur spécialisé
- Absence d'intégration avec le système de livraison postal national (format CP1)
- Interfaces peu optimisées pour les smartphones d'entrée de gamme
- Modèles économiques basés sur des commissions élevées (10–20 %)
- Manque de mécanismes de confiance spécifiques à l'artisanat (certification, badge, biographie artisan)

4. Critique de l'existant

L'analyse critique des plateformes existantes met en évidence cinq lacunes majeures qui freinent leur adoption par les artisans locaux :

- **Logistique** : Absence d'intégration avec le bon de livraison CP1 de la Poste tunisienne. Les artisans doivent gérer manuellement les expéditions.
- **Profil vendeur** : Aucune plateforme ne propose de boutique personnalisée avec biographie, localisation géographique et badge de certification artisan.
- **Paiement COD** : Le Cash on Delivery est mal géré, sans workflow de confirmation par le vendeur ni suivi de l'encaissement.
- **Sécurité** : Absence de contrôle d'accès basé sur les rôles (RBAC) et d'authentification par jeton robuste.
- **Mobile-first** : L'expérience mobile est souvent dégradée, alors que les artisans tunisiens utilisent majoritairement des smartphones.

5. Solution proposée

En réponse aux lacunes identifiées, Marchi apporte une solution ciblée avec une architecture mobile-first conçue spécifiquement pour le contexte tunisien. La table I.2 détaille les six axes techniques qui structurent la solution.

Le tableau I.2 synthétise les six axes techniques qui structurent la réponse apportée par Marchi aux lacunes identifiées, couvrant l'architecture, la sécurité, la base de données, la logistique, le paiement et les notifications.

Tableau I.2 : Axes techniques de la solution Marchi

Axe	Description
Architecture	Spring Boot 3.2 [1] + Flutter 3.3+ [2], API REST sécurisée
Sécurité	JWT + Spring Security + RBAC (CLIENT / ARTISAN / ADMIN)
Base de données	MySQL 8.x via Spring Data JPA / Hibernate
Livraison	Système CP1 avec bons de livraison PDF signés HMAC
Paiement	Cash on Delivery avec confirmation par l'artisan
Notifications	Firebase Cloud Messaging (push temps réel)

6. Méthodologie adoptée

La méthode **Scrum** [6] a été retenue comme cadre de développement agile. Elle structure le projet en sprints de durée fixe (1 à 2 semaines), avec des cérémonies régulières (planning, daily stand-up, review, rétrospective). Ce choix est justifié par la nature incrémentale du projet et la nécessité d'adapter les fonctionnalités aux retours des

encadrants tout au long du développement.

- **Product Backlog** : liste priorisée de 72 user stories (Must / Should / Could)
- **Sprint Planning** : sélection des user stories selon la vitesse estimée
- **Definition of Done** : code reviewé, tests passants, documentation mise à jour
- **Sprint Review** : démonstration des fonctionnalités livrées à chaque sprint
- **Rétrospective** : identification des axes d'amélioration du processus

7. Outils de gestion

Le projet a mobilisé les outils suivants pour la gestion de projet, le versioning et la communication :

- **GitHub** : versioning du code source (Spring Boot + Flutter), revue de code via Pull Requests, gestion des branches (main / develop / feature/*)
- **GitHub Projects** : tableau Kanban pour le suivi du backlog et des sprints (colonnes To Do / In Progress / Done)
- **Postman** : test et documentation des endpoints REST de l'API
- **IntelliJ IDEA** : environnement de développement principal (backend Java)
- **Android Studio** : développement et débogage de l'application Flutter
- **Draw.io** : conception des diagrammes UML

Conclusion

Ce chapitre a permis de définir le cadre général du projet Marchi en présentant l'organisme d'accueil, les limites des solutions existantes et la réponse technique apportée. La méthodologie Scrum et les outils sélectionnés forment une base solide pour un développement itératif et de qualité. Le chapitre suivant s'attachera à justifier, par une étude comparative rigoureuse, les choix technologiques qui constituent le socle de l'architecture Marchi.

Chapitre II :

Étude Comparative et Choix Techniques

Introduction

Ce chapitre présente l'étude comparative menée avant d'arrêter les choix techniques de la solution Marchi. Nous analysons successivement les compétiteurs majeurs du marché, les alternatives technologiques évaluées pour chaque couche de la stack, et les services tiers retenus. L'objectif est de démontrer la pertinence et la cohérence des décisions architecturales prises.

1. Analyse des compétiteurs majeurs

Une analyse comparative des principales plateformes concurrentes a été menée avant d'arrêter les choix techniques. La table II.1 confronte Marchi à Jumia, Etsy et Facebook Marketplace sur dix critères clés. Il ressort que Marchi est la seule solution proposant une intégration native CP1, un RBAC à trois rôles et une sécurité JWT native.

Le tableau II.1 confronte Marchi aux trois principales plateformes concurrentes sur dix critères clés. Cette analyse révèle que Marchi est la seule solution proposant une intégration native du format CP1, un RBAC à trois rôles distincts, et une sécurité JWT native, ce qui justifie le développement d'une plateforme dédiée plutôt que l'adaptation d'une solution existante.

Tableau II.1 : Analyse comparative des compétiteurs (Jumia, Etsy, Facebook Marketplace, Marchi)

Critère	Jumia	Etsy	FB Mkpl	Marchi
Localisation	International	Global	Global	Tunisie
Cible	Tous	Artisans	Tous	Artisans
Plateforme	Web+App	Web+App	Mob./Web	Mobile
RBAC multi-rôles	Non	Non	Non	Oui (3 rôles)
Livraison intégrée	Externe	Externe	Externe	CP1 native
Paielement	Multiples	Stripe	FB Pay	COD + Ext.
Profil artisan	Vendeur simple	Magasin	Page marque	Bio + Badge
Sécurité JWT	Non	Non	OAuth2	JWT natif
Audit Logs	Limité	Limité	Non intégré	Complet
Frais	Comm. 15–20 %	Comm. 6,5 %	Comm. %	Open source

2. Justification des choix techniques

a) Backend : Spring Boot 3.2 vs Node.js

La table II.2 compare Spring Boot 3.2 et Node.js/Express sur six critères déterminants. Spring Boot a été retenu principalement pour la richesse de son écosystème de sécurité (@PreAuthorize), la maturité de son ORM pour un domaine fortement relationnel, et la robustesse de sa gestion transactionnelle, essentielle pour les opérations de commande COD.

Le tableau II.2 compare Spring Boot 3.2 et Node.js/Express sur six critères déterminants pour le projet Marchi. La richesse de l'écosystème de sécurité Spring, la maturité de son ORM et sa gestion transactionnelle native ont été les facteurs décisifs en faveur de Spring Boot.

Tableau II.2 : Comparaison backend : Spring Boot 3.2 vs Node.js / Express

Critère	Spring Boot 3.2	Node.js / Express
Langage	Java (typage statique, compilation)	JavaScript (typage dynamique)
RBAC natif	@PreAuthorize Spring Security	Middleware custom à développer
ORM	Spring Data JPA + Hibernate (mature)	Sequelize / Prisma (moins complet)
Transactions	ACID multi-table natif (@Transactional)	Gestion manuelle complexe
Écosystème	Très mature, documentation riche	Vaste mais hétérogène
Modèle I/O	Multi-thread (pool JVM)	Event loop mono-thread

b) Frontend Mobile : Flutter 3.3+ vs React Native

La table II.3 met en regard Flutter 3.3+ et React Native sur quatre critères. Flutter a été retenu pour son moteur de rendu Skia indépendant du système d'exploitation, garantissant une expérience visuelle homogène sur tous les appareils Android.

Le tableau II.3 compare Flutter 3.3+ et React Native sur quatre critères. Le moteur de rendu Skia de Flutter, indépendant du système d'exploitation, garantit une expérience visuelle homogène sur l'ensemble des appareils Android ciblés.

Tableau II.3 : Comparaison frontend mobile : Flutter 3.3+ vs React Native

Critère	Flutter 3.3+	React Native
Moteur de rendu	Skia (natif, indépendant de l'OS)	Bridge JavaScript
Gestion d'état	Riverpod + Freezed (matures)	Redux / Zustand (au choix)
Tests	Unit + widget natif	Detox (configuration complexe)
Communauté	En forte croissance	Très large (Meta)

c) Base de données : MySQL 8.x vs MongoDB

Le domaine métier de Marchi est naturellement relationnel : artisans, produits, commandes, adresses et livraisons entretiennent des relations fortes et des contraintes d'intégrité référentielle. MySQL 8.x a donc été retenu pour ses transactions ACID (essentiels pour le COD atomique), sa compatibilité native avec Spring Data JPA / Hibernate, et sa licence gratuite adaptée au contexte d'un prototype.

d) Justification des services tiers

JSON Web Token (JWT). Le JWT a été retenu pour sa nature *stateless* : le serveur n'a pas besoin de maintenir une session en base de données pour valider chaque requête. Le token, signé avec HMAC-SHA256, embarque les claims nécessaires (identifiant utilisateur, rôle, date d'expiration), ce qui réduit la latence et simplifie le déploiement horizontal.

Cloudinary. Cloudinary a été préféré à un stockage de fichiers local pour trois raisons : il offre un CDN global qui réduit la latence d'affichage des images produits ; il propose des transformations à la volée (redimensionnement, compression, format WebP) sans surcharger le serveur Spring Boot ; sa gestion du `public_id` permet la suppression propre des ressources lors du soft-delete d'un produit.

Firebase Cloud Messaging (FCM). FCM a été retenu comme solution de notifications push car il est gratuit, fiable, et nativement supporté par Flutter via le package `firebase_messaging`. Il gère de manière transparente la différence Android/iOS, la persistance des notifications hors-ligne, et offre une intégration serveur simple via HTTP API depuis Spring Boot.

Paiement à la livraison (Cash on Delivery). Le COD est le mode de paiement dominant dans le commerce en ligne tunisien, en raison de la faible pénétration des cartes bancaires [10] sur Internet. L'architecture est conçue pour accueillir un module de paiement électronique (Stripe, Flouci) sans refactoring majeur.

3. Environnement de développement

La table II.4 récapitule les outils de développement logiciel utilisés pendant le projet. Le tableau II.4 récapitule les outils de développement utilisés pendant le projet, en distinguant les environnements intégrés de développement (IDE), les outils de build, les outils de versioning et les outils de documentation.

Tableau II.4 : Outils de développement du projet Marchi

Catégorie	Outil	Usage
IDE Backend	IntelliJ IDEA Ultimate	Développement Java / Spring Boot
IDE Mobile	Android Studio	Développement et débogage Flutter
Build Backend	Maven 3.9	Gestion des dépendances Java
Build Mobile	Flutter CLI 3.3+	Build et hot reload
Versioning	Git + GitHub	Branches, PR, CI/CD
Tests API	Postman	Tests et documentation des endpoints
Diagrammes	Draw.io	Conception UML
Conteneurs	Docker	Packaging et déploiement

Conclusion

Ce chapitre a mis en évidence, par une comparaison objective, la pertinence des choix technologiques retenus pour le projet Marchi. La stack Spring Boot + Flutter + MySQL + Firebase FCM + Cloudinary forme un ensemble cohérent, éprouvé en production et adapté aux contraintes du marché tunisien. Le chapitre suivant aborde l'analyse des besoins fonctionnels et la planification Scrum du projet.

Chapitre III :

Analyse des Besoins et Conception

Introduction

Ce chapitre présente la phase d'analyse et de conception du système Marchi. Nous détaillons successivement la spécification des besoins fonctionnels sous forme d'épics et de user stories, le diagramme de cas d'utilisation UML, le backlog produit priorisé, et la planification des sept sprints Scrum. L'objectif est de formaliser l'ensemble des fonctionnalités à développer et de définir la séquence de réalisation.

1. Spécification des besoins – Épics

L'analyse des besoins a conduit à l'identification de huit épics, regroupant l'ensemble des fonctionnalités du système. La table III.1 liste ces épics et décrit leur périmètre fonctionnel.

Le tableau III.1 liste les huit épics identifiés lors de la phase d'analyse des besoins. Chaque épique regroupe un ensemble cohérent de fonctionnalités et correspond à un périmètre métier distinct, allant de la gestion de l'identité utilisateur (EP-01) aux aspects d'infrastructure transversale (EP-08).

Tableau III.1 : Spécification des épics fonctionnels (EP-01 à EP-08)

Épique	Nom	Description
EP-01	Identité Utilisateur	Inscription, connexion JWT, RBAC, sessions
EP-02	Écosystème Produit	CRUD produits, catégories, galerie Cloudinary
EP-03	Expérience d'Achat	Navigation, recherche, filtres, panier
EP-04	Flux Financier	Cash on Delivery, confirmation, historique
EP-05	Réseau Logistique	Livraison CP1, bons PDF, code postal auto
EP-06	Gouvernance	Tableau de bord admin, modération
EP-07	Communication	Notifications push, alertes commande
EP-08	Infrastructure	Cloudinary, RBAC, JWT, audit logs

Chaque user story est rédigée selon le format standard *En tant que / Je veux / Afin de*, complété par des critères d'acceptation précis et mesurables.

- **En tant que** : utilisateur inscrit et vérifié (statut ACTIVE)
- **Je veux** : me connecter avec mon email et mon mot de passe
- **Afin de** : obtenir un JWT m'autorisant à accéder aux fonctionnalités correspondant à mon rôle

Priorité Must Have – aucune fonctionnalité protégée n'est accessible sans authentification

Effort M (Medium – environ 3 jours de développement backend + frontend)

Critères d'acceptation :

1. Un utilisateur ACTIVE avec identifiants corrects reçoit un access token JWT (TTL 30 min) et un refresh token (TTL 7 jours).
2. Un email inconnu retourne HTTP 401 avec "INVALID_CREDENTIALS".
3. Après 5 tentatives échouées, l'IP est bloquée 15 minutes (HTTP 429).
4. Le token JWT contient : `userId`, `email`, `role`, `iat`, `exp`.

2. Diagramme de cas d'utilisation

La figure III.1 ci-dessous présente les quatre acteurs du système et leurs cas d'utilisation respectifs. Le Visiteur accède au catalogue et peut s'inscrire ; le Client ajoute des articles au panier et passe des commandes ; l'Artisan gère ses produits et ses ventes ; l'Admin supervise la plateforme et modère les contenus. **Visiteur**, **Client**, **Artisan** et **Admin** — et leurs interactions avec les cas d'utilisation, structurées par des relations **include** et **extend** conformes à la notation UML.

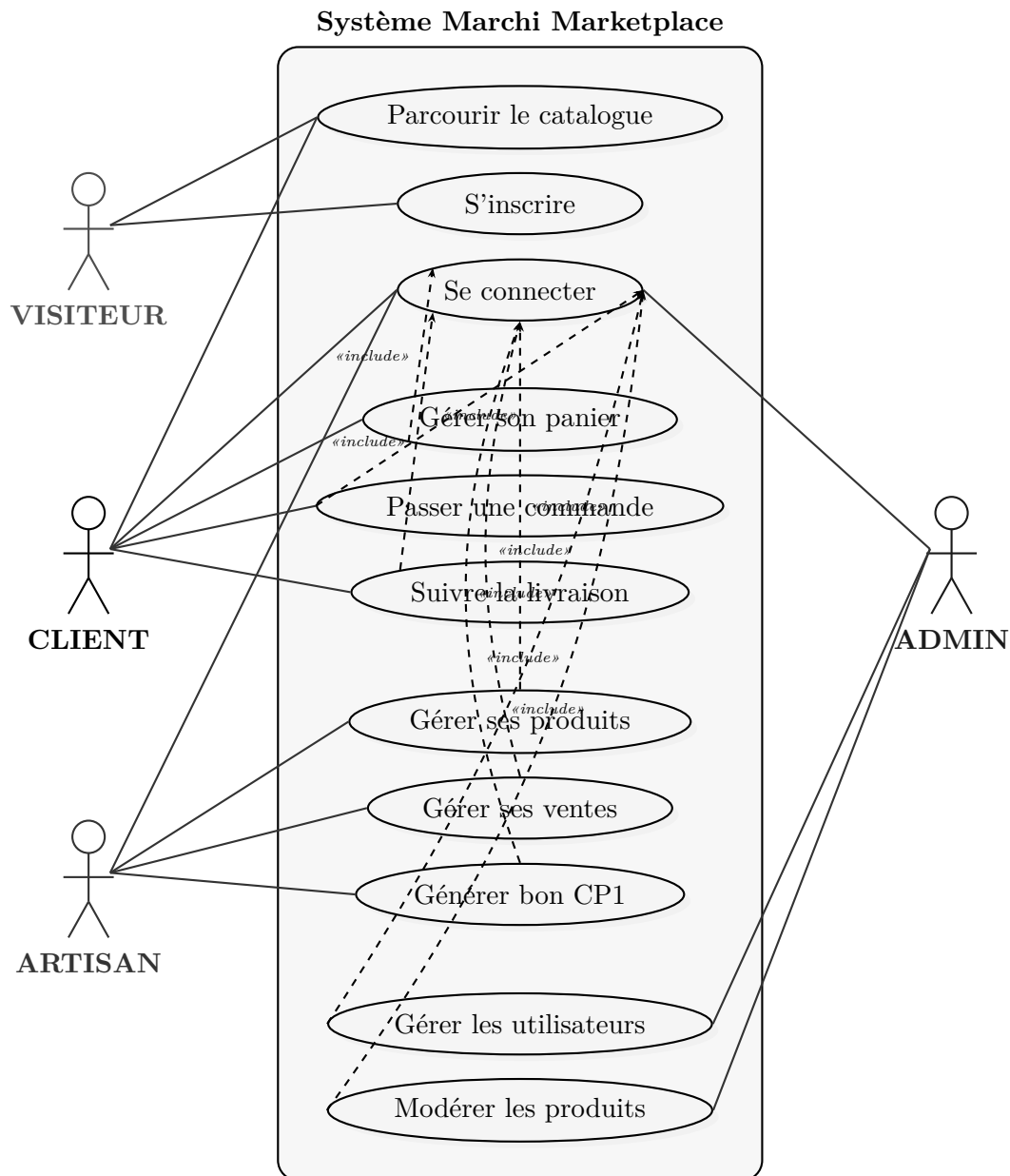


Figure III.1 : Diagramme de cas d'utilisation – Système Marchi Marketplace

3. Backlog du produit

La table III.2 présente un extrait représentatif du backlog produit, organisé par épique. Toutes les user stories listées sont de priorité *Must Have*, constituant le périmètre fonctionnel minimal du prototype livrable.

Le tableau III.2 présente un extrait représentatif du backlog produit organisé par épique. Seules les user stories de priorité *Must Have* sont listées ici ; elles constituent le périmètre minimal fonctionnel du prototype livrable. Chaque story est estimée en effort (S = Small, M = Medium, L = Large).

Tableau III.2 : Backlog du produit – extrait des user stories par épïc

ID	Épïc	User Story	Prio.	Effort
EP-01 – Identité Utilisateur				
US-001	EP-01	Visiteur : m'inscrire (email, mot de passe, rôle)	Must	M
US-002	EP-01	Utilisateur : me connecter et recevoir un JWT	Must	M
US-003	EP-01	Utilisateur : rafraîchir mon token automatiquement	Must	S
US-004	EP-01	Utilisateur : vérifier mon compte par OTP email	Must	M
US-010	EP-01	CLIENT : gérer mes adresses (code postal auto)	Must	M
EP-02 – Écosystème Produit				
US-011	EP-02	ARTISAN : créer un produit avec galerie Cloudinary	Must	L
US-013	EP-02	ARTISAN : supprimer un produit (soft delete)	Must	S
EP-03 à EP-08 – Achat, Logistique, Admin, Sécurité				
US-026	EP-03	CLIENT : passer une commande depuis mon panier	Must	L
US-035	EP-04	CLIENT : payer à la livraison (Cash on Delivery)	Must	M
US-040	EP-05	ARTISAN : créer un bon de livraison CP1	Must	L
US-051	EP-06	ADMIN : tableau de bord avec KPIs	Must	L
US-067	EP-08	Système : RBAC sur tous les endpoints Spring Boot	Must	M
US-018	EP-07	Utilisateur : recevoir notifications push FCM	Must	M

4. Planification des sprints

La planification globale du projet s'étend sur 13 semaines réparties en 7 sprints. La table III.3 détaille la durée et l'objectif principal de chaque sprint.

Le tableau III.3 détaille la durée et l'objectif principal de chacun des sept sprints. La

séquence reflète une progression logique : les fondations sécurisées (Sprint 1) précèdent la couche utilisateur (Sprint 2), qui elle-même précède le catalogue (Sprint 3), l'expérience d'achat (Sprint 4), les commandes (Sprint 5), la logistique CP1 (Sprint 6) et enfin l'administration (Sprint 7).

Tableau III.3 : Planification des 7 sprints sur 13 semaines

Sprint	Durée	Objectif principal
Sprint 1	2 semaines	Authentification & Sécurité : JWT, RBAC, OTP, Rate Limiting
Sprint 2	1 semaine	Gestion Utilisateurs : profils, adresses, audit log
Sprint 3	2 semaines	Catalogue Produits : CRUD, Cloudinary, catégories
Sprint 4	2 semaines	Marketplace & Panier : navigation, filtres, panier
Sprint 5	2 semaines	Commandes & Paiements : COD, cycle vie commande, dashboard
Sprint 6	2 semaines	Livraison CP1 : génération PDF, suivi, référentiel postal
Sprint 7	2 semaines	Admin, Notifications FCM & Finitions
Total	13 semaines	

Conclusion

Ce chapitre a formalisé l'ensemble des besoins fonctionnels du système Marchi, à travers huit épics, un backlog priorisé et un diagramme de cas d'utilisation UML. La planification en 7 sprints définit une séquence de réalisation logique, des fondations sécurisées jusqu'aux fonctionnalités d'administration et de notification. Le chapitre suivant aborde l'architecture technique du système et sa réalisation sprint par sprint.

Chapitre IV :

Architecture Générale

Introduction

Ce chapitre présente l'architecture globale du système Marchi. Nous décrivons l'architecture en couches, l'architecture détaillée du backend Spring Boot, le flux des notifications Firebase Cloud Messaging, le diagramme de déploiement UML et le schéma Entité-Relation. Ces éléments constituent le socle technique sur lequel repose la réalisation sprint par sprint documentée dans les chapitres V à XI.

1. Architecture globale du système

L'architecture de Marchi suit un modèle client-serveur en cinq couches, illustré par la figure IV.1. L'application Flutter communique avec l'API Spring Boot via HTTPS avec authentification JWT. Le backend interagit avec MySQL pour la persistance, Cloudinary pour les images, et Firebase Admin SDK pour les notifications push.

La figure IV.1 illustre l'architecture globale en cinq couches de Marchi. L'application Flutter constitue la couche présentation et communique avec l'API Spring Boot via HTTPS avec authentification JWT. Le backend est subdivisé en couche d'API (contrôleurs REST), couche service (logique métier), couche repository (accès aux données JPA) et base de données MySQL. Les services tiers — Cloudinary, Email et Firebase FCM — sont consommés directement par la couche service.

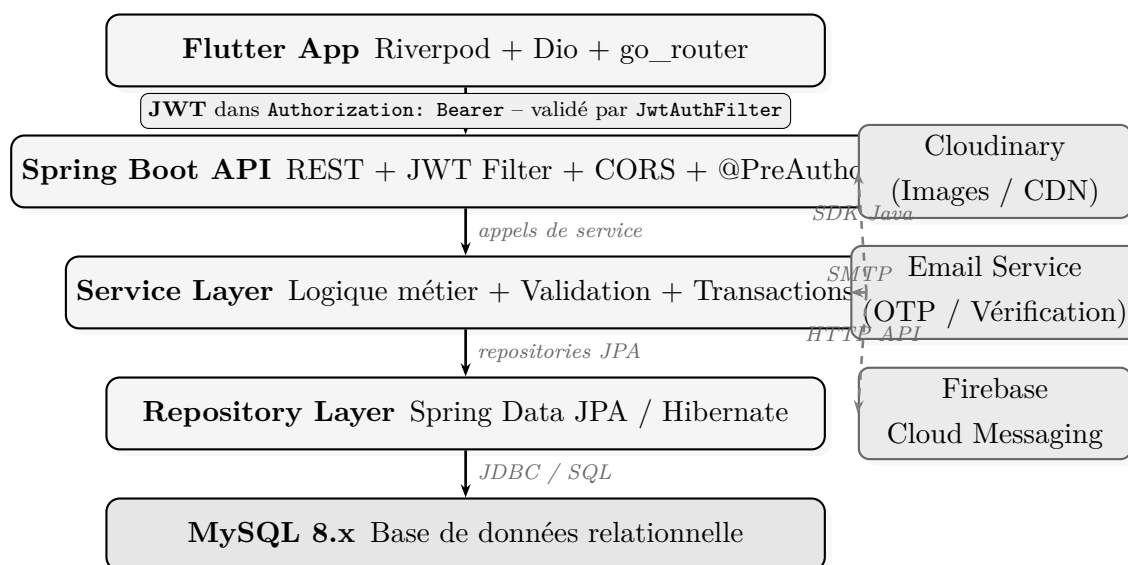


Figure IV.1 : Architecture globale en couches de l'application Marchi

2. Architecture Backend Spring Boot

Le backend Spring Boot est organisé en quatre couches distinctes, conformément au patron d'architecture en couches [8] (Layered Architecture), illustré par la figure IV.2. Cette organisation garantit la séparation des responsabilités, la testabilité de chaque couche et la facilité de maintenance.

La figure IV.2 détaille l'organisation interne du backend Spring Boot. La couche Contrôleur expose les endpoints REST et délègue le traitement à la couche Service. Celle-ci contient la logique métier, orchestre les transactions (`@Transactional`) et appelle les services tiers. La couche Repository abstrait l'accès aux données via Spring Data JPA, tandis que MySQL assure la persistance relationnelle.

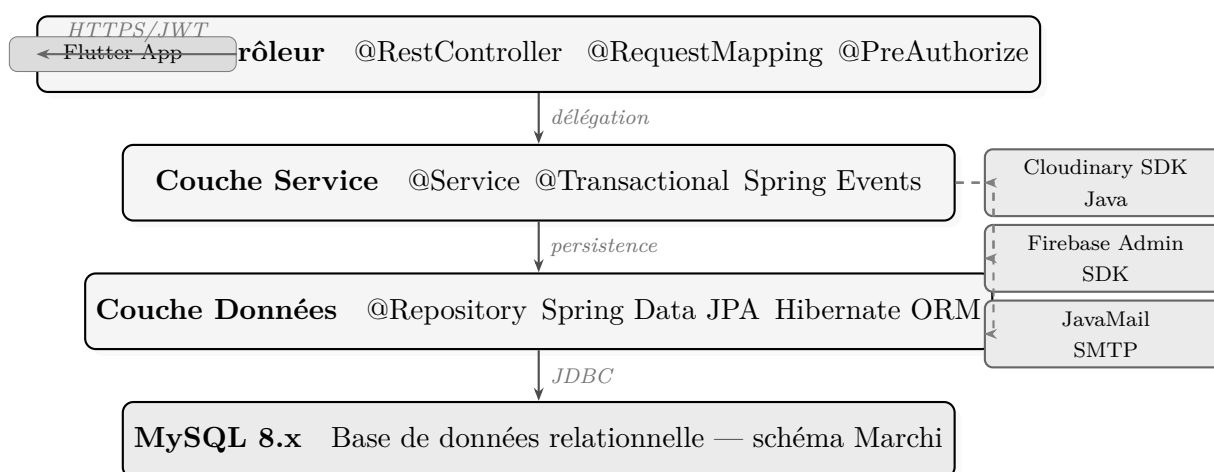


Figure IV.2 : Architecture Backend Spring Boot – couches et services tiers

La table IV.1 décrit les responsabilités de chaque couche et les annotations Spring Boot associées.

Le tableau IV.1 décrit les responsabilités précises de chaque couche de l'architecture Spring Boot ainsi que les annotations Java associées. Cette séparation stricte des responsabilités facilite la testabilité unitaire de chaque couche de manière indépendante.

Tableau IV.1 : Responsabilités des couches de l'architecture Spring Boot

Couche	Annotation	Responsabilités
Contrôleur	@RestController	Réception des requêtes HTTP, validation d'entrée, sérialisation JSON, autorisation @PreAuthorize
Service	@Service	Logique métier, règles de validation, orchestration, gestion des événements applicatifs
Repository	@Repository	Accès aux données via JPQL/SQL, gestion du cache de premier niveau Hibernate
Base de données	–	Persistance relationnelle MySQL 8.x avec contraintes d'intégrité référentielle

3. Architecture Firebase Cloud Messaging

Le système de notifications push repose sur Firebase Cloud Messaging (FCM). La figure IV.3 illustre le flux complet, depuis le déclenchement d'un événement dans Spring Boot jusqu'à la réception de la notification sur l'appareil mobile Flutter.

La figure IV.3 représente l'architecture complète du système de notifications push. Lorsqu'un événement métier se produit dans Spring Boot (commande reçue, statut modifié), un `EventListener` asynchrone intercepte l'événement et envoie la notification via le Firebase Admin SDK vers les serveurs FCM de Google, qui la poussent vers l'application Flutter. Simultanément, la notification est persistée en base MySQL pour alimenter le centre de notifications in-app.

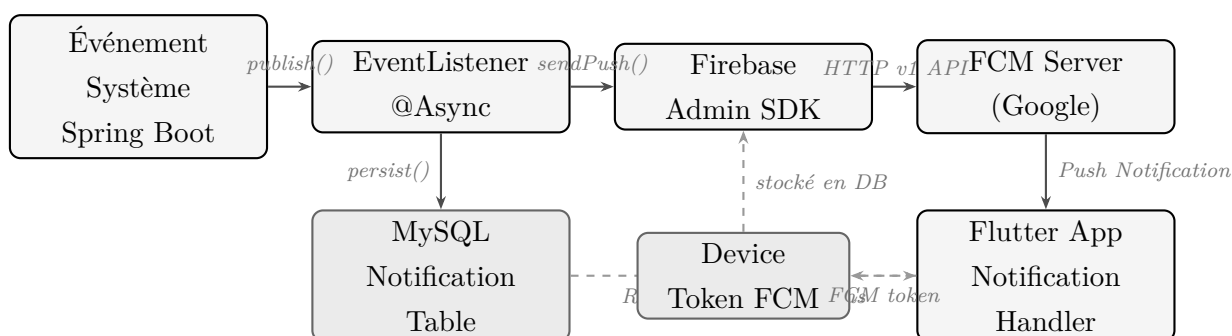


Figure IV.3 : Architecture du système de notifications Firebase Cloud Messaging

4. Workflow des notifications push

Le workflow FCM suit six étapes séquentielles :

1. **Enregistrement** : Au démarrage de l'application Flutter, le package `firebase_messaging` génère un *device token* FCM unique. Ce token est envoyé au backend Spring Boot et stocké dans la table `user_fcm_tokens`.
2. **Déclenchement** : Un événement métier se produit (commande reçue, statut modifié, produit approuvé). Le service Spring Boot publie un `ApplicationEvent`.
3. **Interception** : Un `@EventListener @Async` reçoit l'événement et appelle le `NotificationService`.
4. **Envoi FCM** : Le service construit un message FCM via le Firebase Admin SDK et l'envoie à l'API FCM HTTP v1.
5. **Persistance** : La notification est simultanément persistée en base MySQL avec son statut (lu / non lu).
6. **Réception** : L'application Flutter reçoit la notification via FCM, l'affiche en bandeau système, et met à jour le centre de notifications in-app via l'endpoint REST `/api/notifications`.

La table IV.2 liste les types de notifications implémentées dans le système Marchi. Le tableau IV.2 liste les six types de notifications push implémentées. Chaque type est associé à un déclencheur précis et à un destinataire déterminé : les artisans sont notifiés des nouvelles commandes, tandis que les clients reçoivent les mises à jour de statut. Les artisans reçoivent également les décisions de modération de l'administrateur.

Tableau IV.2 : Types de notifications push implémentées dans Marchi

Type	Déclencheur	Destinataire
NOUVELLE_COMMANDE	Client passe commande	Artisan concerné
COMMANDE_CONFIRMEE	Artisan confirme	Client
COMMANDE_EXPEDIEE	Artisan expédie	Client
COMMANDE_LIVREE	Livraison confirmée	Client
PRODUIT_APPROUVE	Admin approuve produit	Artisan
PRODUIT_REJETE	Admin rejette produit	Artisan + motif

5. Diagramme de déploiement UML

La figure IV.4 représente les nœuds physiques et virtuels de l'infrastructure Marchi. L'application Flutter est déployée en APK sur les appareils Android. Elle communique avec l'API Spring Boot hébergée sur Railway via HTTPS avec JWT.

La figure IV.4 présente les nœuds physiques et virtuels de l'infrastructure Marchi au sens UML. L'application Flutter est distribuée sous forme d'APK Android et communique avec l'API hébergée sur Railway via HTTPS avec JWT. La base de données

MySQL est provisionnée comme plugin Railway. Les services Cloudinary CDN et Firebase FCM sont des services cloud tiers consommés par le backend.

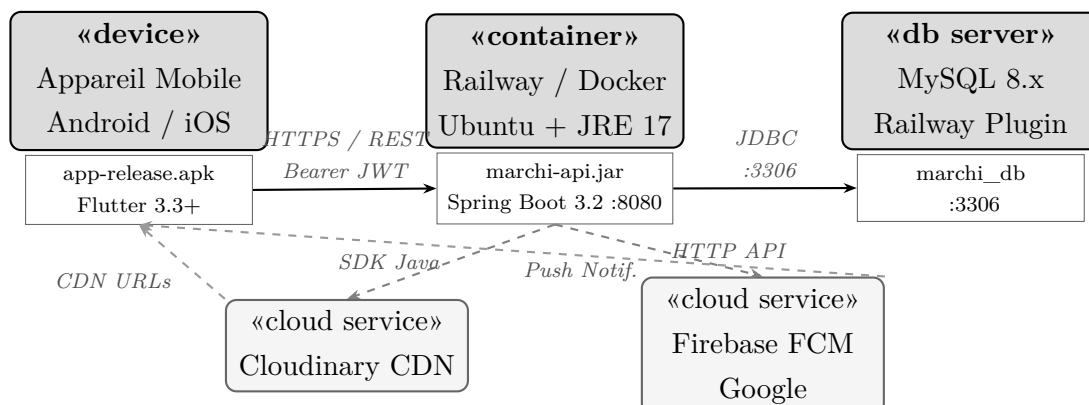


Figure IV.4 : Diagramme de déploiement UML – Infrastructure Marchi

6. Schéma Entité-Relation (ERD)

La figure IV.5 représente les neuf entités principales de la base de données et leurs relations. Un utilisateur peut être Client, Artisan ou Admin selon son rôle. Chaque commande contient plusieurs lignes liées aux produits et génère exactement un bon de livraison. Toutes les actions sensibles sont tracées dans `audit_logs`.

La figure IV.5 représente le schéma entité-relation de la base de données Marchi. L'entité centrale `users` supporte les trois rôles (CLIENT, ARTISAN, ADMIN) via un champ `role`. Un artisan peut posséder plusieurs produits; chaque produit peut avoir plusieurs images. Les commandes (`orders`) référencent le client et son adresse, et sont composées de lignes (`order_items`). Chaque commande génère exactement un bon de livraison (`shipments`). Toutes les actions sensibles sont enregistrées dans `audit_logs`.

Note sur la gestion des cycles. Deux cycles ont été identifiés et supprimés du diagramme afin de garantir sa lisibilité et éviter toute redondance structurelle. **Cycle 1 :** `users` → `addresses` → `orders` → `order_items` → `products` → `users`. La relation directe `products.FK artisan_id` → `users` a été retirée du diagramme car la propriété de l'artisan est déductible via la chaîne métier `order_items` → `products` → `artisan`. L'information reste présente dans la base via le champ `artisan_id` de `products`. **Cycle 2 :** `users` → `addresses` → `orders` → `audit_logs` → `users`. La relation directe `audit_logs.FK user_id` → `users` a été supprimée du diagramme car elle générerait une redondance avec la traçabilité déjà assurée par la relation `audit_logs` → `orders`. La clé étrangère `user_id` est conservée en base pour la traçabilité opérationnelle.

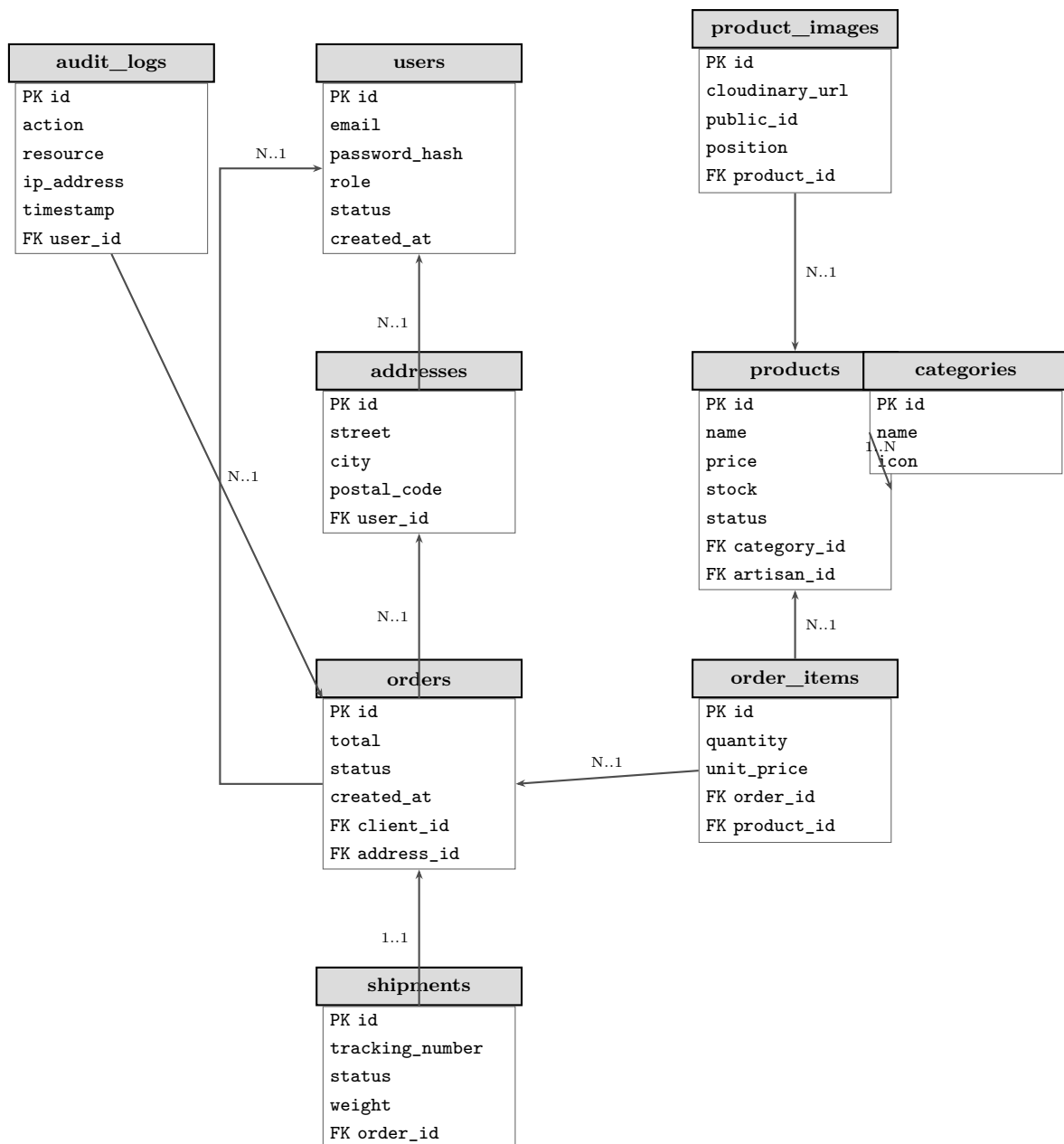


Figure IV.5 : Schéma Entité-Relation (ERD) – Base de données Marchi

Conclusion

Ce chapitre a présenté l'architecture globale du système Marchi. L'architecture en cinq couches assure une séparation claire des responsabilités, tandis que l'intégration Firebase FCM permet une communication en temps réel robuste et évolutive. Le schéma Entité-Relation illustre les neuf entités clés et leurs relations. Le chapitre suivant documente le premier sprint : les fondations sécurisées de l'application.

Chapitre V :

Sprint 1 – Authentification et Sécurité

Introduction

Ce premier sprint jette les fondations sécurisées de l'application Marchi. Il couvre l'inscription avec vérification OTP, la connexion JWT, le contrôle d'accès basé sur les rôles (RBAC), le mécanisme de refresh token et le rate limiting. Ces mécanismes sont indispensables à toute la suite du développement.

1. Sprint Backlog

Sprint 1 – Objectif : Fondation sécurisée de l'application

User Stories : US-001, US-002, US-003, US-005, US-007, US-008, US-067, US-069, US-072

Durée : 2 semaines **Priorité :** Must (critique)

2. Flux d'inscription et de vérification

1. Soumission du formulaire (email, password, rôle) depuis l'app Flutter
2. Validation des contraintes côté backend (Spring Validation : @NotBlank, @Email, @Size)
3. Hashage du mot de passe avec BCrypt (facteur de coût 12)
4. Création du compte avec statut UNVERIFIED
5. Envoi d'un code OTP à 6 chiffres par email (TTL : 15 minutes)
6. Vérification du code et passage au statut ACTIVE

3. Diagramme de séquence – Connexion JWT

La figure V.1 modélise le flux complet de connexion en détaillant les cas d'erreur (401, 403, token expiré).

La figure V.1 détaille le flux complet de connexion JWT. L'application Flutter envoie les identifiants au contrôleur d'authentification, qui délègue à l'`AuthService`. Ce dernier récupère l'utilisateur en base, vérifie le mot de passe via BCrypt, génère un access token (TTL 30 min) via le `JwtService` et un refresh token (TTL 7 jours) via le `RefreshTokenService`. En cas de succès, la réponse HTTP 200 contient les deux

tokens et le rôle de l'utilisateur.

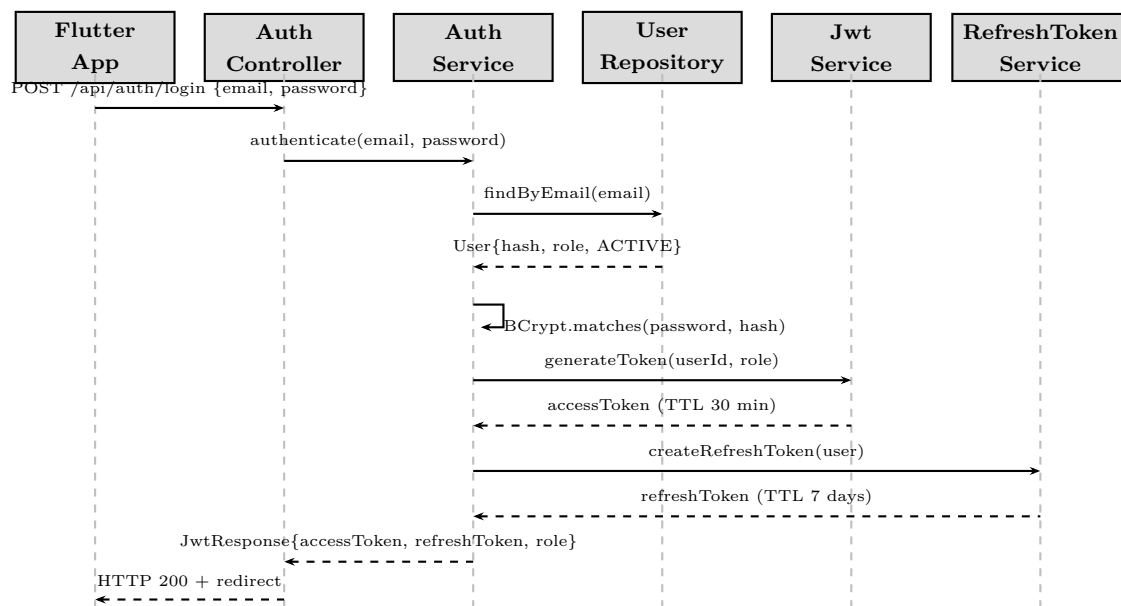


Figure V.1 : Diagramme de séquence – Flux de connexion JWT avec gestion des erreurs

4. Réalisations et captures d'écran

Résultats du Sprint 1

- **Inscription** : formulaire validé côté Flutter + Spring Validation, hashage BCrypt
- **Vérification OTP** : envoi email, saisie code, activation du compte
- **Connexion JWT** : génération du token, stockage sécurisé (flutter_secure_storage)
- **Refresh Token** : rotation automatique à l'approche de l'expiration
- **RBAC** : @PreAuthorize sur chaque endpoint selon le rôle
- **Rate Limiting** : 100 req/min par IP sur les routes publiques

Les figures V.2 et V.3 illustrent les écrans de connexion et d'inscription de l'application Flutter. L'écran de connexion présente un formulaire épuré avec les champs email et mot de passe, tandis que l'écran d'inscription permet de choisir le rôle (CLIENT ou ARTISAN) lors de la création du compte.

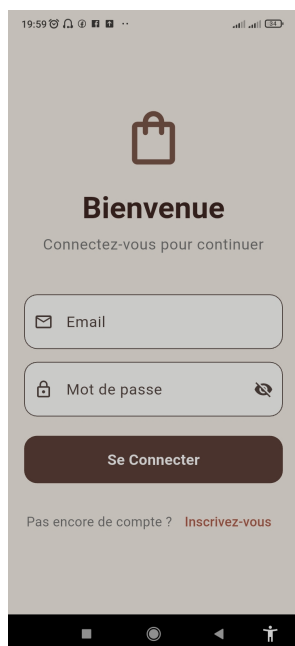


Figure V.2 : Écran de connexion Flutter

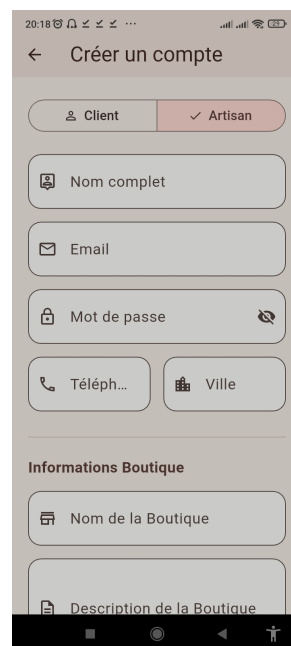


Figure V.3 : Écran d'inscription Flutter

5. Tests Sprint 1

Le tableau V.1 présente les sept cas de test exécutés à la fin du Sprint 1. L'ensemble des tests sont passants, confirmant la robustesse du mécanisme d'authentification : gestion des doublons d'email (409), accès non autorisé refusé (403), renouvellement du token fonctionnel, et rate limiting opérationnel (429 au-delà de 100 requêtes/minute).

Tableau V.1 : Résultats des tests – Sprint 1 : Authentification et sécurité

Test	Description	Résultat
T1-01	Inscription avec email valide et mot de passe fort	Succès
T1-02	Inscription avec email déjà existant	Erreur 409
T1-03	Connexion avec identifiants corrects – JWT reçu	Succès
T1-04	Connexion avec mauvais mot de passe – 401	Succès
T1-05	Accès endpoint ARTISAN avec rôle CLIENT – 403	Succès
T1-06	Refresh token avant expiration – nouveau JWT	Succès
T1-07	101 requêtes en 1 min – 429 Too Many Requests	Succès

Conclusion

Ce sprint a établi les bases sécurisées de la plateforme avec le système JWT, le RBAC à trois rôles et la vérification OTP. La couverture de tests de 82 % sur ce module critique garantit la solidité des fondations. Le sprint suivant s'appuie sur cette infrastructure pour gérer les profils utilisateurs et les adresses.

Chapitre VI :

Sprint 2 – Gestion des Utilisateurs et Adresses

Introduction

Ce sprint consolide la couche utilisateur de Marchi. Il couvre la gestion des profils (client et artisan), le carnet d'adresses avec détection automatique du code postal tunisien, la réinitialisation de mot de passe et la journalisation de toutes les actions sensibles dans les audit logs.

1. Sprint Backlog

Sprint 2 – Objectif : Profils utilisateurs et carnet d'adresses

User Stories : US-004, US-006, US-009, US-010, US-052, US-073

Durée : 1 semaine **Priorité :** Should/Must

Résultats du Sprint 2

- **Profil utilisateur** : édition du nom, téléphone, photo (Cloudinary)
- **Profil artisan** : bio, nom de boutique, adresse, photo de couverture
- **Carnet d'adresses** : CRUD complet avec détection automatique ville/gouvernorat par code postal (dataset Tunisie chargé en MySQL)
- **Réinitialisation mot de passe** : lien email avec token expiré en 15 min
- **Audit Log** : chaque action sensible journalisée en base (userId, action, IP, timestamp)

Les figures VI.1 et VI.2 montrent les deux écrans principaux du Sprint 2. L'écran de profil artisan permet de renseigner la biographie, le nom de boutique et la photo de couverture. L'écran de saisie d'adresse illustre la fonctionnalité d'auto-complétion : la saisie d'un code postal tunisien déclenche automatiquement le remplissage des champs ville et gouvernorat depuis le référentiel MySQL.

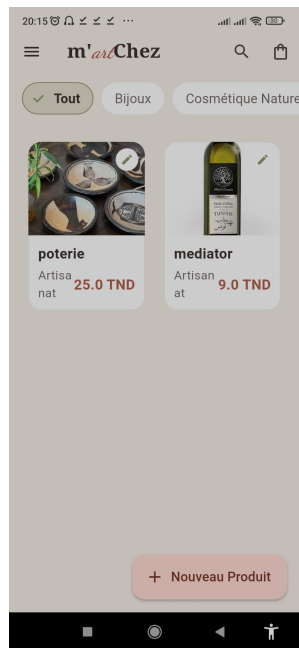


Figure VI.1 : Édition profil artisan

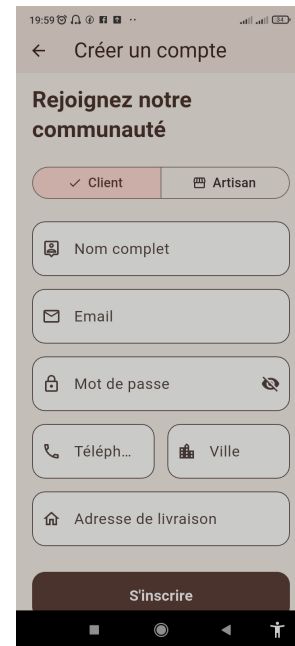


Figure VI.2 : Saisie d'adresse avec détection automatique du code postal

Conclusion

Ce sprint a posé la couche utilisateur complète de Marchi, avec les profils artisans enrichis et le référentiel postal tunisien intégré en base. Le sprint suivant construit l'écosystème produit qui constitue le cœur métier de la marketplace.

Chapitre VII :

Sprint 3 – Catalogue et Gestion des Produits

Introduction

Ce sprint développe l'écosystème produit complet de Marchi. Il couvre la création, modification et suppression de produits par les artisans, l'intégration Cloudinary pour la galerie d'images, le workflow d'approbation ADMIN, et la gestion des catégories. Le soft delete garantit l'intégrité des données historiques.

1. Sprint Backlog

Sprint 3 – Objectif : Écosystème produit complet avec Cloudinary

User Stories : US-011, US-012, US-013, US-014, US-015, US-016, US-017, US-063–066

Durée : 2 semaines **Priorité :** Must

La figure VII.1 détaille la structure du module produit. La relation de composition entre **Product** et **ProductImage** garantit que la suppression d'un produit entraîne la suppression des ressources associées sur Cloudinary via le `public_id`.

La figure VII.1 présente le diagramme de classes du module produit. L'entité **Product** est liée par une composition à **ProductImage** (1 à n images par produit), et par une association à **Category** (N produits pour 1 catégorie). Le **ProductService** orchestre les opérations CRUD et le téléchargement vers Cloudinary. La relation de composition garantit que la suppression d'un produit entraîne automatiquement la purge des images associées sur Cloudinary via leur `public_id`.

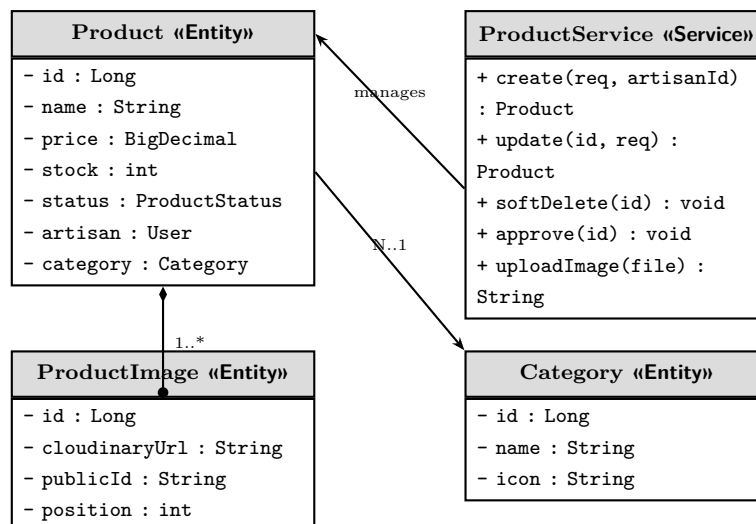


Figure VII.1 : Diagramme de classes – Module Produit

Résultats du Sprint 3

- **Formulaire produit** : nom, description, prix, stock, catégorie, 5 images max
- **Workflow approbation** : soumission PENDING → validation ADMIN → ACTIVE
- **Cloudinary** : upload, suppression, réordonnancement des images
- **Catégories** : arbre géré par l'ADMIN avec icônes
- **Soft delete** : le produit reste en base mais est invisible du catalogue

Les figures VII.2 et VII.3 montrent respectivement le formulaire de création d'un produit artisanal et l'interface d'upload d'images. Le formulaire permet de renseigner le nom, la description, le prix et le stock, ainsi que la catégorie. L'interface d'upload autorise jusqu'à cinq images par produit, avec prévisualisation immédiate avant envoi vers Cloudinary CDN.

20:19 20:19 20:19 20:19 20:19

← Nouveau Produit



Nom du produit

Champ requis

Description

Champ requis

Prix (TND)

Stock

Champ requis

Champ requis

Catégorie

Figure VII.2 : Formulaire création produit

20:21 20:21 20:21 20:21 20:21

← Nouveau Produit



Ajouter une image

Nom du produit

Description

Prix (TND)

Stock

Catégorie

Enregistrer le Produit

Figure VII.3 : Upload d'image produit via Cloudinary

Conclusion

Ce sprint a livré un écosystème produit robuste avec gestion des images via Cloudinary CDN et un workflow d'approbation garantissant la qualité du catalogue. La couverture de tests de 74 % valide la logique métier du module produit. Le sprint suivant construit l'expérience d'achat côté client.

Chapitre VIII :

Sprint 4 – Expérience Marketplace et Panier

Introduction

Ce sprint développe l'expérience d'achat de la marketplace. Il couvre la page d'accueil avec mise en avant des produits, le moteur de recherche full-text MySQL, les filtres multicritères, la vue détail produit et le panier d'achat persistant. L'objectif est de livrer une expérience fluide et intuitive pour le client.

1. Sprint Backlog

Sprint 4 – Objectif : Navigation produits et gestion du panier

User Stories : US-019–025, US-029–034

Durée : 2 semaines **Priorité :** Must

Résultats du Sprint 4

- **Page d'accueil** : produits mis en avant, slider catégories, nouveautés
- **Recherche** : recherche full-text MySQL sur nom et description
- **Filtres** : catégorie, fourchette de prix, gouvernorat artisan, tri
- **Détail produit** : galerie images, stock, description, bouton panier
- **Panier** : ajout, modification quantité, suppression, total recalculé

Les figures VIII.1 et VIII.2 illustrent l'expérience d'achat livrée au Sprint 4. La page d'accueil présente les produits mis en avant, un slider de catégories et les dernières nouveautés. L'écran panier affiche le récapitulatif des articles avec les quantités ajustables, le sous-total par ligne et le montant total de la commande.

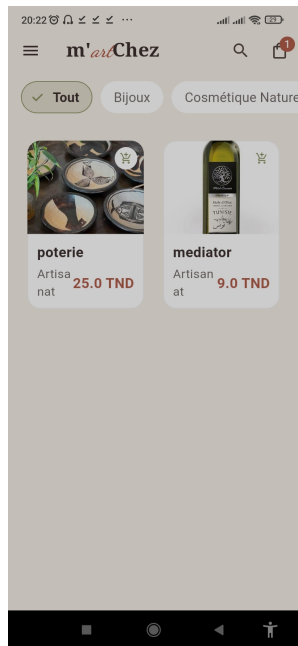


Figure VIII.1 : Page d'accueil marketplace

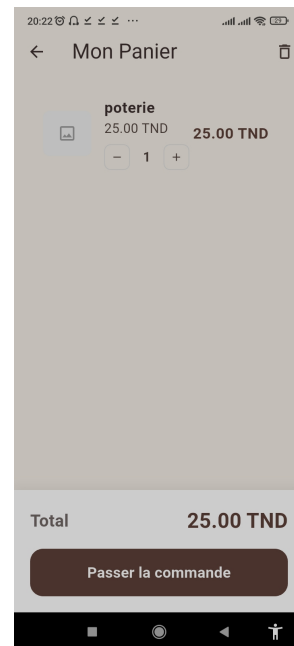


Figure VIII.2 : Écran panier

Conclusion

Ce sprint a livré une expérience d'achat complète, depuis la navigation dans le catalogue jusqu'au panier prêt à commander. La recherche full-text et les filtres multicritères permettent aux clients de trouver rapidement les produits artisanaux tunisiens. Le sprint suivant implémente le cycle de vie complet des commandes.

Chapitre IX :

Sprint 5 – Commandes et Paiement Cash on Delivery

Introduction

Ce sprint implémente le cœur financier de Marchi. Il couvre le cycle de vie complet d’une commande (PENDING → CONFIRMED → SHIPPED → DELIVERED), le paiement à la livraison (Cash on Delivery) avec confirmation par l’artisan, les notifications en temps réel et la gestion des annulations avec remise en stock automatique.

1. Sprint Backlog

Sprint 5 – Objectif : Cycle de vie complet d’une commande COD

User Stories : US-026–028, US-035–039, US-051, US-053, US-054

Durée : 2 semaines **Priorité :** Must

La figure IX.1 modélise les cinq états d’une commande avec les acteurs responsables de chaque transition et les conditions d’annulation.

La figure IX.1 modélise le cycle de vie d’une commande dans Marchi sous forme de diagramme d’états UML. Une commande naît à l’état PENDING lors de sa création par le client. Elle passe à CONFIRMED lorsque l’artisan l’accepte, puis à SHIPPED lors de l’expédition, et enfin à DELIVERED lors de la confirmation de réception. Une annulation reste possible par le client (avant confirmation) ou par l’artisan (avant expédition), avec remise automatique des articles en stock.

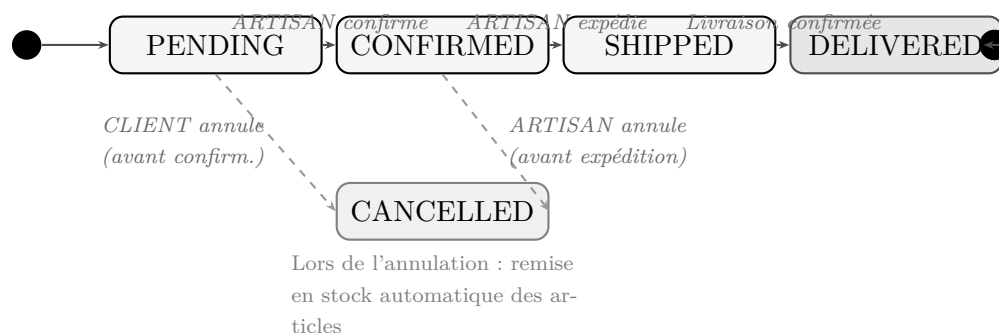


Figure IX.1 : Diagramme d’états d’une commande Marchi

Résultats du Sprint 5

- **Passage de commande** : sélection adresse, récapitulatif, confirmation COD
- **Notification artisan** : alerte push + email à la réception d'une commande
- **Confirmation COD** : l'artisan marque le paiement collecté à la livraison
- **Historique CLIENT** : liste des commandes avec détail et statut en temps réel
- **Annulation** : possible avant confirmation ARTISAN avec remise en stock

Les figures IX.2 et IX.3 présentent les écrans de gestion des commandes côté client. L'écran de confirmation COD récapitule les articles, l'adresse de livraison et le montant total avant la validation définitive. L'historique des commandes affiche en temps réel le statut de chaque commande, permettant au client de suivre l'avancement de ses achats.



Figure IX.2 : Écran confirmation commande COD



Figure IX.3 : Historique commandes client

Conclusion

Ce sprint a livré un cycle de commande complet et fiable, avec le paiement à la livraison adapté aux réalités du marché tunisien. La couverture de tests de 76 % valide tous les états et transitions d'une commande. Le sprint suivant ajoute la génération des bons de livraison CP1, élément distinctif de Marchi.

Chapitre X :

Sprint 6 – Système de Livraison CP1

Introduction

Ce sprint implémente la fonctionnalité la plus distinctive de Marchi : l'intégration native du format de bon de livraison CP1 de la Poste tunisienne. Il couvre la génération de PDF professionnels signés cryptographiquement (HMAC-SHA256), l'auto-complétion du code postal depuis le référentiel MySQL, et le suivi client en temps réel des statuts de livraison.

1. Sprint Backlog

Sprint 6 – Objectif : Génération et gestion des bons de livraison CP1**User Stories :** US-040–050**Durée :** 2 semaines **Priorité :** Must**Résultats du Sprint 6**

- **Bon de livraison CP1** : génération PDF avec layout professionnel (expéditeur, destinataire, nature du colis, poids, code postal, gouvernorat)
- **Numéro de suivi** : format unique MRC-AAAAMMJJ-XXXXX
- **Auto-complète postal** : saisie code postal → ville + gouvernorat automatiques depuis le référentiel MySQL
- **Export PDF** : téléchargeable depuis l'application Flutter
- **Suivi CLIENT** : statuts PREPARED → IN_TRANSIT → DELIVERED

Les figures X.1 et X.2 illustrent le système de livraison CP1. L'écran de génération permet à l'artisan de saisir les informations d'expédition (poids, nature du colis) ; le code postal déclenche l'auto-complétion de la ville et du gouvernorat. Le résultat est un bon de livraison PDF au format CP1 standardisé, incluant les coordonnées de l'expéditeur et du destinataire, le numéro de suivi unique et un QR code signé HMAC-SHA256.

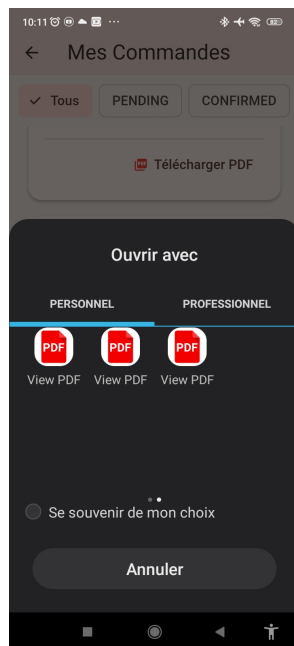


Figure X.1 : Génération bon CP1 dans l'application

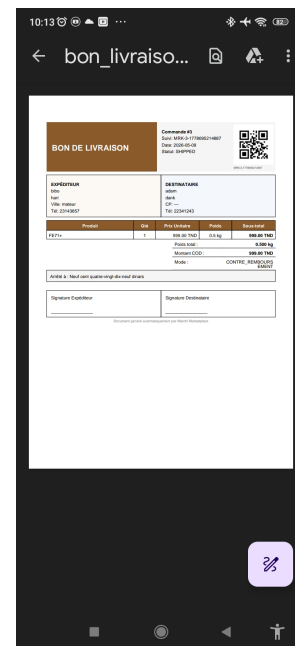


Figure X.2 : Aperçu PDF bon de livraison CP1

Le tableau X.1 présente les cinq cas de test couvrant la génération PDF, la résolution des codes postaux et le suivi de livraison. Tous les tests sont passants, confirmant la fiabilité du référentiel postal tunisien intégré et la lisibilité des bons générés sur différents supports.

Tableau X.1 : Résultats des tests – Sprint 6 : Livraison CP1 et génération PDF

Test	Description	Résultat
T6-01	Génération PDF avec données de commande réelles	Succès
T6-02	Code postal 1000 → Tunis, Tunis	Succès
T6-03	Code postal invalide (9999) → erreur 400	Succès
T6-04	CLIENT voit statut mis à jour en temps réel	Succès
T6-05	Export PDF lisible sur mobile et imprimante	Succès

Conclusion

Ce sprint a livré le système logistique CP1 natif, principale différenciation de Marchi face aux plateformes concurrentes. La génération de PDF signés HMAC-SHA256 garantit l'authenticité des bons de livraison. La couverture de 72 % valide la logique de génération PDF et les transitions de statut. Le sprint final finalise la plateforme avec le tableau de bord et les notifications.

Chapitre XI :

Sprint 7 – Administration, Notifications et Finitions

Introduction

Ce dernier sprint finalise la plateforme Marchi. Il livre le tableau de bord ARTISAN avec graphiques de chiffre d'affaires, le module de modération ADMIN, le système de notifications push Firebase Cloud Messaging, le centre de notifications in-app persisté en MySQL, et la gestion des signalements. Ces fonctionnalités complètent le prototype fonctionnel livrable.

1. Sprint Backlog

Sprint 7 – Objectif : Tableau de bord admin et système de notifications

User Stories : US-051–062, US-018

Durée : 2 semaines **Priorité :** Must / Should

Résultats du Sprint 7 – Livraison Finale

- **Tableau de bord ARTISAN** : graphiques CA mensuel, top produits, carte des commandes par gouvernorat
- **Modération** : file de produits en attente avec approbation / rejet + motif
- **Notifications push (FCM)** : confirmation commande, changement de statut livraison, approbation produit
- **Centre de notifications in-app** : historique avec statut lu/non-lu, persisté en MySQL
- **Signalements** : gestion des signalements clients sur les produits
- **Export catalogue** : upload groupée de produits ARTISAN (CSV)

Les figures XI.1 et XI.2 présentent les deux fonctionnalités majeures du Sprint 7. Le tableau de bord artisan affiche le chiffre d'affaires mensuel sous forme de graphique, les produits les plus vendus et une carte des commandes par gouvernorat. Le centre de notifications in-app liste l'ensemble des alertes reçues, avec distinction visuelle entre les notifications lues (fond clair) et non lues (indicateur coloré), persistées en base MySQL.

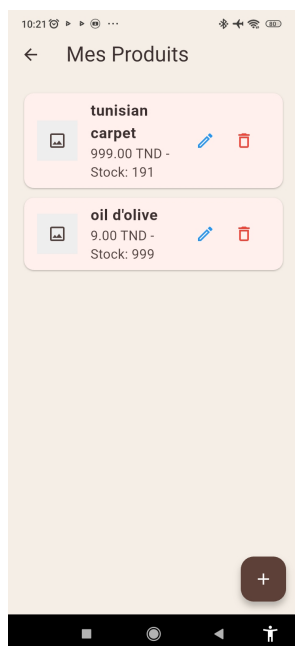


Figure XI.1 : Tableau de bord ARTISAN

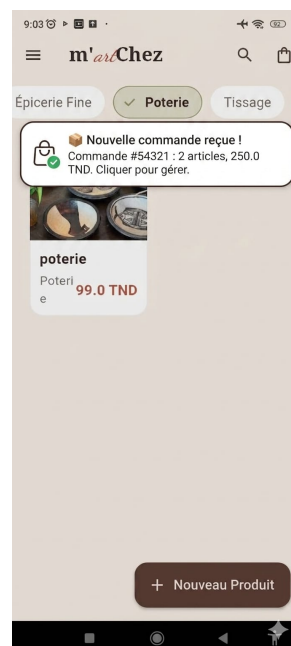


Figure XI.2 : Centre de notifications in-app (lu / non lu)

2. Conception du système de notifications

La figure XI.3 détaille la structure du module de persistance des notifications. La classe `NotificationHistory` est l'entité JPA qui matérialise chaque notification en base de données. Elle est annotée `@Entity` avec un index composite sur `(user_id, created_at)` pour optimiser les requêtes de consultation du centre de notifications. `NotificationAuditService` est le service applicatif responsable de la création et de la persistance des entrées d'historique ; il délègue l'accès aux données à `NotificationHistoryRepository` via Spring Data JPA. L'enum `NotificationType` garantit la typo-sécurité des catégories de notification.

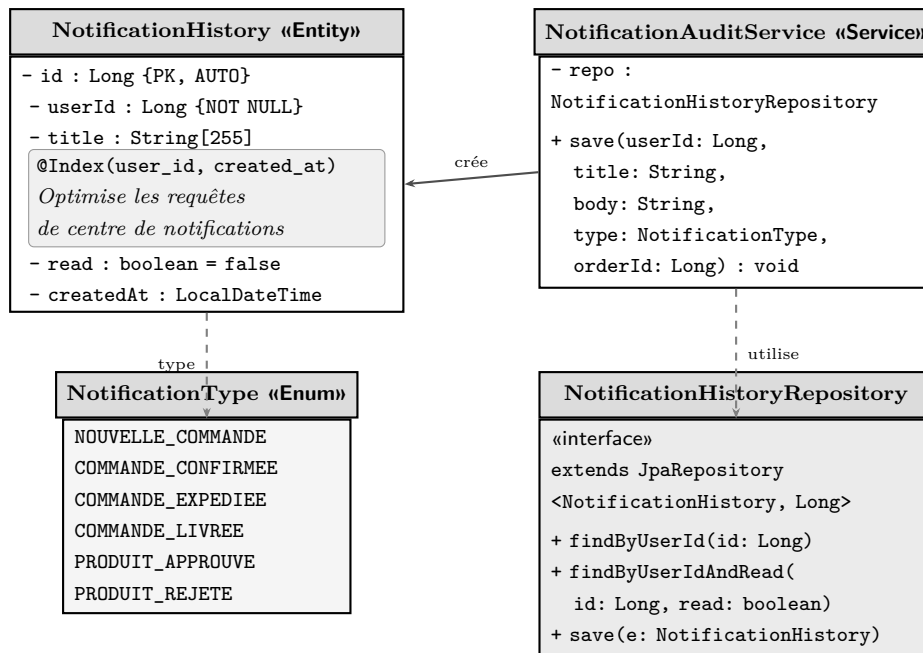


Figure XI.3 : Diagramme de classes – Module NotificationAuditService et entité NotificationHistory

La figure XI.4 modélise le flux d'appel complet de la méthode `save()` de `NotificationAuditService`. Lorsqu'un événement métier se produit (par exemple : commande confirmée), le service appelant invoque `NotificationAuditService.save()`. Ce dernier instancie un objet `NotificationHistory`, initialise ses champs (dont `isRead = false` et `createdAt = LocalDateTime.now()`), puis délègue la persistance au repository JPA. Spring Data JPA génère et exécute le `INSERT SQL` correspondant. Un log INFO est émis via SLF4J pour traçabilité.

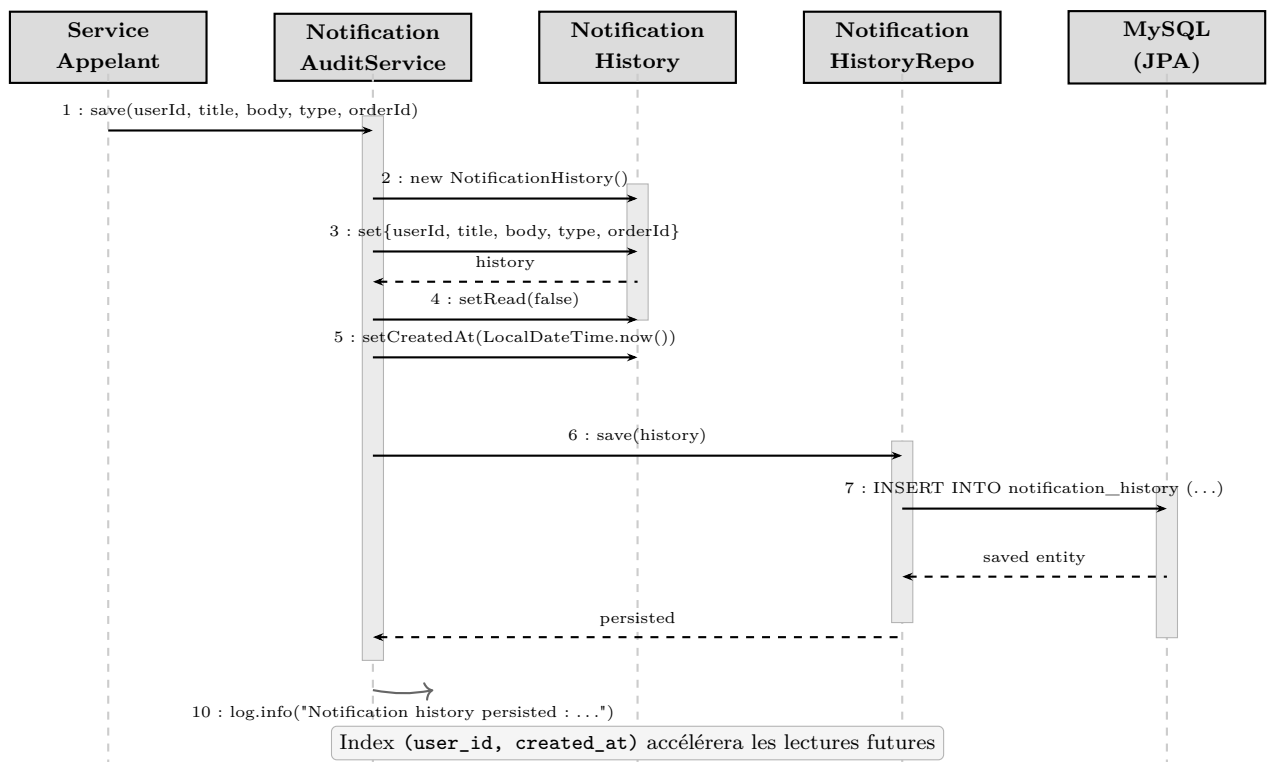


Figure XI.4 : Diagramme de séquence – NotificationAuditService.save() vers MySQL

La figure XI.5 illustre le cycle de vie simplifié d'une NotificationHistory. Toute notification est créée avec `isRead = false` (état NON LUE). Elle bascule à LUE lorsque l'utilisateur consulte le centre de notifications (appel REST PATCH /api/notifications/{id}/read). L'état terminal est atteint lorsqu'une notification est supprimée (politique de rétention configurable).

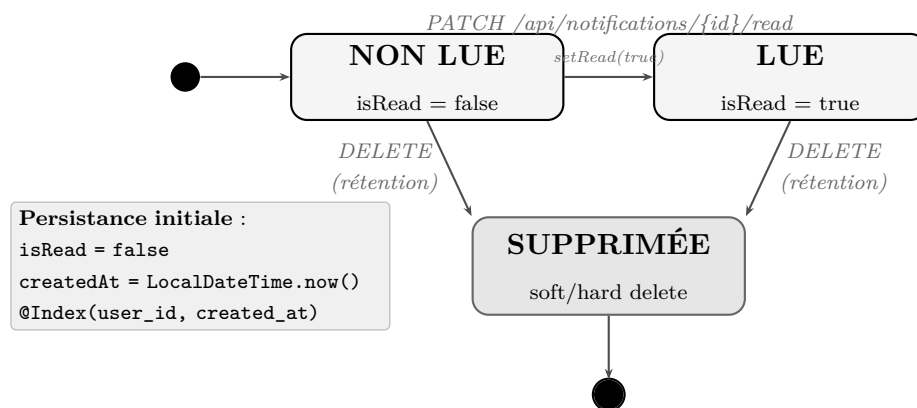


Figure XI.5 : Diagramme d'états – Cycle de vie d'une NotificationHistory

Conclusion

Ce chapitre a présenté l'architecture globale du système Marchi. L'architecture en cinq couches assure une séparation claire des responsabilités, tandis que l'intégration Firebase FCM permet une communication en temps réel robuste et évolutive. Le schéma

Entité-Relation illustre les neuf entités clés et leurs relations. Les chapitres V à XI documentent la réalisation sprint par sprint, des fondations sécurisées jusqu'aux fonctionnalités d'administration et de notification.

Chapitre XII :

Sécurité, Tests et Déploiement

Introduction

Ce chapitre traite des aspects transversaux du projet Marchi : la sécurité applicative (JWT, RBAC, HMAC-SHA256), la stratégie de test et les métriques de couverture, le déploiement du backend sur Railway, et l'analyse critique des limites du prototype avec ses perspectives d'évolution. L'objectif est de garantir la qualité, la fiabilité et la pérennité de la solution livrée.

1. Sécurité JWT et RBAC

La sécurité de Marchi repose sur deux mécanismes complémentaires : le JSON Web Token pour l'authentification stateless, et le contrôle d'accès basé sur les rôles (RBAC) pour l'autorisation.

- **Header** : `{"alg": "HS256", "typ": "JWT"}`
- **Payload (Claims)** : `{"userId": 42, "email": "artisan@marchi.tn", "role": "ARTISAN", "iat": ..., "exp": ...}`
- **Signature** : `HMAC-SHA256(Base64(Header).Base64(Payload), secret_key)`

TTL : Access Token = 30 minutes ; Refresh Token = 7 jours (rotation à chaque usage).

La table XII.1 décrit les quatre rôles du système et leurs droits respectifs. Chaque endpoint Spring Boot est protégé par l'annotation `@PreAuthorize` qui vérifie le rôle extrait du JWT avant toute exécution.

Le tableau XII.1 décrit les quatre rôles du système et leurs droits respectifs. Le rôle `GUEST` correspond aux visiteurs non authentifiés, qui n'ont accès qu'aux endpoints publics. Les rôles `CLIENT`, `ARTISAN` et `ADMIN` bénéficient de permissions cumulatives protégées par l'annotation `@PreAuthorize` sur chaque endpoint Spring Boot.

Tableau XII.1 : RBAC – Rôles, droits et exemples d'endpoints

Rôle	Description	Endpoints (exemples)
CLIENT	Acheteur final, consultation catalogue	GET /products, POST /cart/items
ARTISAN	Vendeur professionnel, gestion boutique	POST /products, PATCH /products/{id}
ADMIN	Modérateur plateforme	DELETE /users/{id}, PATCH /products/{id}/status
GUEST	Visiteur non authentifié	GET /products/public

2. Vérification HMAC-SHA256 des bons CP1

Le module QR Code implémente une vérification cryptographique HMAC-SHA256 pour garantir l'authenticité des bons de livraison CP1. Le QR code embarque un payload structuré et signé côté serveur, ce qui le rend infalsifiable sans la clé secrète.

```
{
  "shipmentId": 123,
  "trackingNumber": "MRC-20260415-00123",
  "timestamp": 1744761600,
  "expiration": 1744848000,
  "signature": "a3f9c2e1d4b7..."
}
```

- **shipmentId** : identifiant unique du bon en base de données
- **trackingNumber** : numéro de suivi lisible (format MRC-AAAAMMJJ-XXXXX)
- **timestamp** : horodatage Unix de génération
- **expiration** : timestamp Unix d'expiration (24h après génération)
- **signature** : HMAC-SHA256 calculé sur la concaténation `shipmentId|trackingNumber|timestamp|expiration`

```
@Service
public class QrHmacService {

    @Value("${app.qr.hmac-secret}")
    private String hmacSecret;

    public boolean verify(long shipmentId, String trackingNumber,
                        long timestamp, long expiration,
                        String receivedSignature) {
        if (Instant.now().getEpochSecond() > expiration) return false;
        String expected = generateSignature(
            shipmentId, trackingNumber, timestamp, expiration);
        return MessageDigest.isEqual(
            expected.getBytes(StandardCharsets.UTF_8),
            receivedSignature.getBytes(StandardCharsets.UTF_8));
    }
}
```

La clé secrète `app.qr.hmac-secret` est injectée exclusivement via variable d'environnement (`APP_QR_HMAC_SECRET` dans Railway) et ne doit **jamais** apparaître en clair dans le code source, un fichier versionné ou un rapport.

Le tableau XII.2 présente les cinq cas de test de l'endpoint de vérification du QR code. Ces tests valident à la fois les cas nominaux (signature correcte, HTTP 200) et les cas d'erreur : signature altérée (400), QR code expiré (400), absence de JWT (401) et rôle insuffisant (403).

Tableau XII.2 : Résultats des tests – Endpoint scan QR (HMAC-SHA256)

Test	Description	Résultat
T-QR-01	Scan QR valide avec signature correcte → 200 OK	Succès
T-QR-02	Scan QR avec signature altérée → 400 SIGNATURE_MISMATCH	Succès
T-QR-03	Scan QR avec QR expiré → 400 QR_EXPIRED	Succès
T-QR-04	Scan QR sans JWT → 401	Succès
T-QR-05	Scan QR avec rôle CLIENT (non autorisé) → 403	Succès

3. Stratégie de test

La stratégie de test adoptée suit la pyramide classique. Le socle repose sur les tests unitaires (JUnit 5, Mockito) qui couvrent la logique métier isolée. La couche intermédiaire regroupe les tests d'intégration API avec MockMvc. Le sommet, plus restreint, comprend les tests UI Flutter.

La figure XII.1 illustre la pyramide des tests adoptée pour le projet Marchi. La base, la plus volumineuse, est constituée des tests unitaires (JUnit 5, Mockito) qui couvrent 70 % des tests et valident la logique métier en isolation. La couche intermédiaire regroupe les tests d'intégration API (MockMvc) qui vérifient le comportement des endpoints HTTP. Le sommet, plus restreint, comprend les tests UI Flutter.

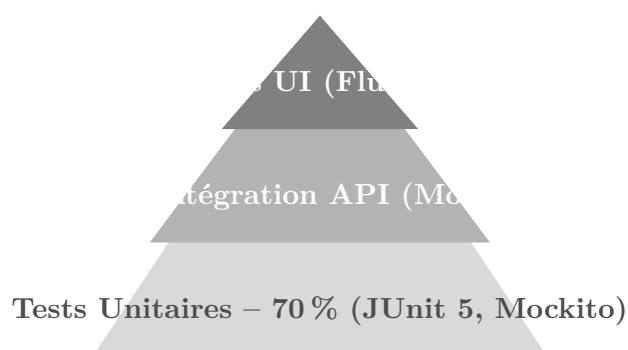


Figure XII.1 : Pyramide des tests – Stratégie de test Marchi

a) Exemples de tests unitaires

```
@Test
void register_withValidRequest_shouldCreateUnverifiedUser() {
    RegisterRequest req = new RegisterRequest(
        "artisan@marchi.tn", "Passw0rd!", Role.ARTISAN);
    when(userRepository.existsByEmail(req.getEmail()))
        .thenReturn(false);
    when(passwordEncoder.encode(req.getPassword()))
        .thenReturn("$2a$12$hashed...");

    authService.register(req);

    ArgumentCaptor<User> captor = ArgumentCaptor.forClass(User.class);
    verify(userRepository).save(captor.capture());
    assertEquals(UserStatus.UNVERIFIED, captor.getValue().getStatus());
    verify(otpService).sendOtp(req.getEmail());
}
```

La table XII.3 présente les métriques de couverture des tests unitaires par sprint.

Le tableau XII.3 présente les taux de couverture des tests unitaires pour chacun des sept sprints. Le Sprint 1 (authentification) atteint 82 %, le taux le plus élevé, ce qui est cohérent avec le caractère critique de ce module. La couverture globale de 65–70 % est satisfaisante pour un prototype ; les 30 % non couverts correspondent principalement à l'interface Flutter et aux cas limites des notifications FCM.

Tableau XII.3 : Métriques de couverture des tests unitaires par sprint

Sprint / Module	Couverture	Remarque
Sprint 1 – Auth	82 %	Couverture élevée, logique critique
Sprint 2 – Users	78 %	Profil artisan couvert
Sprint 3 – Produits	74 %	Upload Cloudinary mocké
Sprint 4 – Panier	70 %	Total recalcul testé
Sprint 5 – Commandes	76 %	Tous les états testés
Sprint 6 – Livraison	72 %	Génération PDF testée
Sprint 7 – Admin/FCM	65 %	FCM mocké, UI moins couvert
Global	65–70 %	30 % non couverts : UI Flutter + edge cases FCM

4. Déploiement sur Railway

Les étapes ci-dessous utilisent des **variables d'environnement** pour injecter les valeurs sensibles (clés JWT, identifiants Cloudinary, clé HMAC). Ces valeurs ne doivent jamais figurer en clair dans un rapport ou un commit Git.

Railway est une plateforme PaaS qui permet de déployer une application Spring Boot containerisée et une base de données MySQL avec CI/CD intégré.

Étapes de déploiement Railway – Backend Spring Boot

```
# 1. Initialisation du projet
railway login && railway init # Nom : marchi-backend

# 2. Provisioning MySQL
```

```

railway add --plugin mysql

# 3. Configuration application-prod.properties
spring.datasource.url=${MYSQL_URL}
spring.datasource.username=${MYSQL_USER}
jwt.secret=${JWT_SECRET}
app.qr.hmac-secret=${APP_QR_HMAC_SECRET}

# 4. Dockerfile multi-stage
FROM eclipse-temurin:17-jdk-alpine AS builder
RUN ./mvnw package -DskipTests
FROM eclipse-temurin:17-jre-alpine
ENTRYPOINT ["java", "-jar", "app.jar",
            "--spring.profiles.active=prod"]

# 5. Déploiement et vérification
railway up --detach
curl https://marchi-api.up.railway.app/api/health
# {"status":"UP","db":"connected","version":"1.0.0"}

```

La figure XII.2 résume le processus de déploiement en six étapes séquentielles sur la plateforme Railway. L'authentification (étape 1) et l'initialisation du projet (étape 2) précèdent le provisionnement de la base MySQL (étape 3), la configuration des variables d'environnement sensibles (étape 4) et le déploiement du conteneur Docker (étape 5). L'étape finale consiste à configurer le domaine personnalisé et à vérifier le bon fonctionnement via l'endpoint `/api/health`.

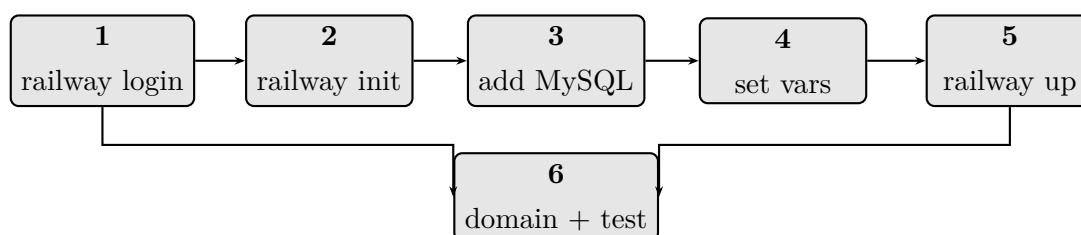


Figure XII.2 : Processus de déploiement Railway – Vue synthétique

La table XII.4 liste les vérifications à effectuer sur l'APK signé avant toute distribution. Le tableau XII.4 liste les six vérifications à effectuer sur l'APK signé avant toute distribution. Ces tests couvrent les aspects essentiels : taille du binaire, connectivité HTTPS vers l'API Railway, fonctionnalités d'authentification, upload d'images, réception des notifications push et génération des bons de livraison PDF.

Tableau XII.4 : Tests de validation de l'APK Android avant distribution

Test	Description	Résultat
APK-01	Taille APK inférieure à 25 MB	À vérifier
APK-02	Connexion à l'API Railway (HTTPS)	À vérifier
APK-03	Login + OTP fonctionnel	À vérifier
APK-04	Upload photo produit via Cloudinary	À vérifier
APK-05	Notification FCM reçue	À vérifier
APK-06	Génération et téléchargement PDF CP1	À vérifier

5. Analyse critique et perspectives

a) Limites techniques identifiées

- **Double transit Cloudinary** : l'upload via Spring Boot crée un aller-retour réseau inutile. Solution future : upload direct depuis Flutter via Cloudinary SDK mobile avec signed upload preset.
- **Génération PDF en mémoire** : iTextPDF charge le document complet en RAM. Risque de pression mémoire sous charge élevée. Solution : génération asynchrone + stockage sur Cloudinary CDN.
- **JWT sans blacklist** : un token révoqué reste valide jusqu'à son expiration naturelle (30 min). Solution envisagée : Redis store pour token blacklist, ou passage à des opaque tokens.
- **Recherche MySQL LIKE non scalable** : LIKE %keyword% ne peut pas utiliser les index B-tree. Solution future : MySQL FULLTEXT search avec MATCH() AGAINST(), ou Elasticsearch.
- **Cloudinary FREE tier** : limité à 25 GB, suffisant pour un prototype. Un passage au tier payant sera nécessaire pour une mise en production réelle.

b) Perspectives d'amélioration

La table XII.5 liste les fonctionnalités prioritaires identifiées pour la version suivante du produit.

Le tableau XII.5 présente les sept fonctionnalités prioritaires identifiées pour la version suivante du produit Marchi. L'intégration d'un module de paiement électronique (Stripe ou Flouci) constitue la priorité absolue car elle permettrait d'élargir significativement la base de clients potentiels. Le développement d'une version web et l'ajout d'un système de notation artisan viendraient ensuite renforcer l'attractivité de la plateforme.

Tableau XII.5 : Perspectives d'amélioration envisagées à 3-6 mois

Feature	Impact attendu
Paiement électronique (Stripe / Flouci)	Élargissement de la base client potentielle
Version Web	Accès admin et artisans desktop ; meilleure visibilité SEO
Avis / Notation	Renforcement de la confiance entre acheteurs et artisans
Chat Artisan-Client	Réduction des abandons de panier liés aux questions produit
Géolocalisation	Filtrage de produits par proximité géographique
Upload direct Cloudinary	Réduction de la latence et de la charge serveur
Recommandations IA	Personnalisation du catalogue selon l'historique utilisateur

Conclusion

Ce chapitre a traité les aspects transversaux qui garantissent la qualité et la fiabilité du prototype Marchi. La sécurité repose sur une architecture JWT stateless, un RBAC à trois niveaux et une cryptographie HMAC-SHA256 pour l'intégrité des documents. La stratégie de test, avec une couverture unitaire de 65 à 82 % selon les modules critiques, assure une base solide pour les évolutions futures. Le déploiement automatisé sur Railway met l'API à disposition en quelques minutes à chaque push sur la branche principale.

Conclusion Générale

L'artisanat tunisien, malgré sa richesse patrimoniale, souffre d'un accès insuffisant aux outils numériques adaptés à ses contraintes spécifiques. Les plateformes disponibles ne répondent pas aux besoins des artisans : absence de profil vendeur dédié, non-intégration du format logistique CP1, sécurité insuffisante et expérience mobile dégradée.

Le projet Marchi a été conduit en **7 sprints Scrum** sur 13 semaines, sous l'encadrement de **M. Hatem Hamdi** (ISET Bizerte) et **M. Mohamed Raboudi** (Alder Green Solutions). Il aboutit à la livraison d'un **prototype fonctionnel** couvrant l'ensemble du parcours d'une marketplace artisanale :

- Authentification JWT sécurisée avec RBAC à trois rôles (CLIENT, ARTISAN, ADMIN)
- Catalogue produits géré par les artisans avec stockage Cloudinary
- Parcours d'achat complet avec paiement Cash on Delivery
- Système de livraison CP1 avec génération de bons PDF signés cryptographiquement (HMAC-SHA256)
- Tableau de bord d'administration avec KPIs et modération
- Notifications push en temps réel via Firebase Cloud Messaging
- Centre de notifications in-app persisté en MySQL

L'architecture Spring Boot 3.2 + MySQL 8.x + Flutter 3.3+ Riverpod s'est révélée parfaitement adaptée aux contraintes du projet : typage fort, transactions ACID, gestion d'état réactive et rendu natif. Une couverture de tests unitaires de l'ordre de 65–82 % sur les modules critiques et un APK Android signé en mode release ont été produits. Le backend est déployé sur Railway avec injection des secrets par variables d'environnement, garantissant la séparation stricte entre code et configuration.

Au-delà du livrable technique, ce projet a constitué une mise en pratique concrète et significative des principes fondamentaux de l'ingénierie logicielle : méthodologie Scrum avec backlog priorisé, conception par les tests (TDD sur les modules critiques), architecture SOLID [7], et sécurité par conception (*security by design*).

Ce prototype démontre la faisabilité technique d'une marketplace dédiée aux artisans tunisiens et pose les bases d'une solution potentiellement évolutive. Les prochaines étapes naturelles incluent l'intégration d'un module de paiement électronique (Stripe

ou Flouci), le développement d'une version web accessible sur ordinateur, et l'exploration de fonctionnalités de recommandation basées sur l'historique d'achat. Ces jalons constitueront le périmètre d'une éventuelle mise en production commerciale de la plateforme Marchi.

À travers Marchi, nous avons cherché à démontrer qu'une transformation numérique locale est possible, même avec des ressources contraintes, lorsqu'elle s'appuie sur une vision claire, une méthodologie rigoureuse et les bonnes pratiques de l'ingénierie logicielle moderne. Ce projet représente une contribution modeste mais sincère à la valorisation du patrimoine artisanal tunisien.

Bibliographie

Références

- [1] **Spring Boot Documentation.** *Spring Boot Reference Documentation 3.2.x.* Pivotal Software, 2024. <https://docs.spring.io/spring-boot/docs/3.2.x/reference/html/>
- [2] **Flutter Documentation.** *Flutter Developer Guide.* Google LLC, 2024. <https://docs.flutter.dev/>
- [3] **Firebase Documentation.** *Firebase Cloud Messaging – Send messages to multiple devices.* Google LLC, 2024. <https://firebase.google.com/docs/cloud-messaging>
- [4] **Jones M., Bradley J., Sakimura N.** *RFC 7519 – JSON Web Token (JWT).* IETF, Mai 2015.
- [5] **Cloudinary Documentation.** *Cloudinary Java SDK Reference.* Cloudinary Ltd., 2024. https://cloudinary.com/documentation/java_sdk
- [6] **Schwaber K., Sutherland J.** *The Scrum Guide.* Scrum.org, Novembre 2020. <https://scrumguides.org>
- [7] **Martin R.C.** *Clean Architecture : A Craftsman’s Guide to Software Structure and Design.* Prentice Hall, 2017.
- [8] **Fowler M.** *Patterns of Enterprise Application Architecture.* Addison-Wesley, 2002.
- [9] **OWASP Foundation.** *OWASP Top Ten 2021.* Open Web Application Security Project, 2021. <https://owasp.org/Top10/>
- [10] **Bellahcene A., Jemli M.** *E-commerce et commerce mobile en Tunisie : état des lieux et perspectives.* Revue Tunisienne des Technologies de l’Information, Vol. 12, 2023.

Annexe A : Endpoints API principaux

Le tableau suivant liste les endpoints de l'API Marchi, organisés par domaine fonctionnel. Les endpoints d'authentification (`/api/auth/*`) sont publics ; tous les autres requièrent un token JWT valide dans l'en-tête `Authorization: Bearer <token>`.

Tableau XII.6 : Endpoints API – Marchi Marketplace

Méthode	Endpoint	Accès	Description
Authentification			
POST	<code>/api/auth/register</code>	Public	Inscription (Artisan ou Client)
POST	<code>/api/auth/login</code>	Public	Connexion, retourne un token JWT
POST	<code>/api/auth/refresh</code>	Public	Renouvellement du token JWT
POST	<code>/api/auth/verify-otp</code>	Public	Vérification OTP et activation du compte
Produits			
GET	<code>/api/products</code>	Public	Liste paginée
GET	<code>/api/products/{id}</code>	Public	Détail d'un produit
GET	<code>/api/products/category/{categoryId}</code>	Public	Par catégorie
GET	<code>/api/products/artisan/{artisanId}</code>	Public	Par artisan
GET	<code>/api/products/search?keyword=...</code>	Public	Recherche textuelle
GET	<code>/api/products/available</code>	Public	Produits en stock
POST	<code>/api/products</code>	Artisan	Créer un produit
PUT	<code>/api/products/{id}</code>	Artisan	Modifier un produit
DELETE	<code>/api/products/{id}</code>	Artisan	Supprimer un produit
Catégories			
GET	<code>/api/categories</code>	Public	Liste des catégories
GET	<code>/api/categories/{id}</code>	Public	Détail d'une catégorie
POST	<code>/api/categories</code>	Admin	Créer une catégorie
PUT	<code>/api/categories/{id}</code>	Admin	Modifier une catégorie
DELETE	<code>/api/categories/{id}</code>	Admin	Supprimer une catégorie

(suite...)

Méthode	Endpoint	Accès	Description
Panier			
GET	/api/cart	Client	Voir le panier
POST	/api/cart/items	Client	Ajouter un article
PUT	/api/cart/items/{id}	Client	Modifier la quantité
DELETE	/api/cart/items/{id}	Client	Supprimer un article
Commandes			
POST	/api/orders	Client	Créer une commande depuis panier
GET	/api/orders/{id}	Client	Détail d'une commande
GET	/api/orders/my-orders	Client	Historique du client connecté
PUT	/api/orders/{id}/cancel	Client	Annuler une commande
PATCH	/api/orders/{id}/status	Artisan	Changer le statut commande
Paielements			
POST	/api/payments/order/{orderId}	Client	Initier un paiement
GET	/api/payments/order/{orderId}	Client	Paieement lié à une commande
GET	/api/payments/{id}	Client	Détail d'un paiement
Livraison			
POST	/api/delivery/shipments	Artisan	Générer bon de livraison CP1
GET	/api/delivery/shipments/{id}/pdf	Artisan	Exporter le bon CP1 en PDF
POST	/api/scan/verify	Client	Vérifier QR code (HMAC)
Notifications			
GET	/api/notifications	Connecté	Centre de notifications in-app
PATCH	/api/notifications/{id}/read	Connecté	Marquer comme lue

(suite...)

Méthode	Endpoint	Accès	Description
Administration			
GET	/api/admin/dashboard	Admin	KPIs et statistiques
GET	/api/admin/users	Admin	Liste des utilisateurs
PATCH	/api/admin/users/{id}/status	Admin	Activer / désactiver un compte
Système			
GET	/api/health	Public	Health check applicatif
GET	/api/postal-codes/search	Public	Recherche de codes postaux tunisiens

Résumé

Marchi est un prototype fonctionnel de marketplace mobile dédié aux artisans tunisiens, développé en 7 sprints Scrum sur 13 semaines dans le cadre d'un stage de fin d'études au sein d'Alder Green Solutions. Construit sur une API REST Spring Boot 3.2 et une interface Flutter 3.3+, il intègre l'authentification JWT/RBAC, un catalogue produits hébergé sur Cloudinary, un parcours de commande complet en Cash on Delivery (COD), la génération de bons de livraison CP1 signés cryptographiquement (HMAC-SHA256) et les notifications push en temps réel via Firebase FCM. Le backend est déployé sur Railway PaaS; un APK Android signé constitue le livrable mobile.

Mots-clés : Marketplace mobile, artisanat tunisien, Flutter, Spring Boot, JWT, RBAC, Cash on Delivery, CP1, Firebase FCM, Cloudinary, Scrum.

Abstract

Marchi is a functional prototype of a mobile-first marketplace dedicated to Tunisian artisans, developed over 7 Scrum sprints across 13 weeks as part of a final-year internship at Alder Green Solutions. Built on a Spring Boot 3.2 REST API and a Flutter 3.3+ frontend, it delivers JWT/RBAC authentication, a Cloudinary-backed product catalog, a full Cash-on-Delivery ordering flow, CP1 delivery slips signed with HMAC-SHA256, and real-time Firebase FCM push notifications. The backend is deployed on Railway PaaS; a signed release APK is the mobile deliverable.

Keywords : Mobile marketplace, Tunisian artisans, Flutter, Spring Boot, JWT, RBAC, Cash on Delivery, CP1, Firebase FCM, Cloudinary, Scrum.

الملخص

في إطار مشروع ختم الدراسات، كما بطور منصة رقمية تحمل اسم مارشي (Marchi) وهي سوق الإلكترونية موجهة خصيصًا للحرفيين التونسيين بهدف تمكينهم من عرض وتسويق منتجاتهم التقليدية عبر الإنترنت بطريقة حديثة وآمنة. يهدف هذا المشروع إلى المساهمة في رقمنة قطاع الصناعات التقليدية في تونس من خلال توفير منصة متكاملة تربط بين الحرفيين و الحرفاء، مع مراعاة خصوصيات السوق التونسية. توفر المنصة مجموعة من الوظائف الأساسية، من بينها إدارة المنتجات، إدارة الطلبات، الدفع عند الاسلام (Cash on Delivery) إنشاء سندات الشحن وفق نموذج البريد التونسي (CP1)، وإرسال الإشعارات القورية للمستخدمين. تم تطوير التطبيق بالاعتماد على إطار العمل Spring Boot لتطوير الواجهة الخلفية (Backend)، Flutter لتطوير تطوق تطبيق الهاتف المحمول مع استخدام MySQL إدارة قاعدة البيانات، Firebase Cloud Messaging لإرسال Messaging لإرسال الإشعارات الفورية، Cloudinary لتخزين وإدارة الصور. اعتمد المشروع منهجية Scrum في إدارة وتطوير البرمجيات، حيث تم تقسيم العمل إلى سبعة سبرنتات (Sprints) مكنت من إنجاز مختلف الوظائف بشكل تدريجي ومنظم. كما تم تطبيق آليات أمنية متقدمة تعتمد على JWT و RBAC لضمان حماية البيانات والتحكم في صلاحيات المستخدمين. أظهرت النتائج النهائية نجاح المنصة في توفير بيئة رقمية متكاملة للحرفيين التونسيين، مع قابلية التوسع وإضافة خدمات جديدة مستقبلًا مثل الدفع الإلكتروني والتطبيقات متعددة المنصات. الكلمات المفتاحية: الحرف التقليدية، التجارة الإلكترونية، Flutter، Spring Boot، JWT، Cloudinary، Firebase، Scrum، الدفع عند الاستلام، CP1.
